FINTECHATHON 2026 · INTERNATIONAL TRACK · TOPIC F — AUTONOMOUS ON-CHAIN WALLET

# QED.

## An AI wallet that moves money only when it can prove it should.

### Technical documentation — architecture, algorithms and security design

QED treats the AI as compromised by default and lets money move only on proofs that Ethereum checks: an owner's mandate times an independent co-signer's certificate (authority), a ring-fenced liabilities invariant (solvency), an exact optimality certificate for paid work and for multilateral netting, proven in public or in zero knowledge (correctness), and a worst case computed on-chain (bounded loss). Two contracts put money on the theorems. This document specifies the system, proves the properties it claims and maps each one to tests and public transactions.

**Live on Ethereum Sepolia**
123 public transactions in three runs · 12 contracts

**33 attacks, 33 reverts**
2,000 tUSD of bounties unclaimed

**Re-verifiable by anyone**
104 Solidity + 50 TypeScript tests · `npm run verify`

Source, tests and evidence: github.com/Nightcore709/qed-agent-wallet
Live dashboard (reads the chain): nightcore709.github.io/qed-agent-wallet
Built from commit `804a1a3628b3` · 2026-10-20 · testnet prototype, not externally audited

# Contents


## Part 0 — Overview

- The idea in seven sentences
- The four outcomes — each one a public transaction
- What makes this different
- Certified clearing: a whole agent economy settles by proof
- QED Private: paid for a proof about data nobody saw
- Theorems with money on them
- How the evidence was produced (full disclosure)
- Status

## Part 1 — System Architecture

- 1. One picture
- 2. Components
- 3. Key lifecycle flows
- 4. Data & evidence model
- 5. Execution & deployment topology
- 6. Gas profile on Sepolia (canonical run)

## Part 2 — Core Algorithms

- 1. The solvency invariant (ring-fenced liabilities)
- 2. Authorisation: owner mandate × independent certificate
- 3. Provable blast radius (worst-case loss under total hot-key compromise)
- 4. Certified optimality: paying AI for proofs, not for answers
- 5. The certifying solver (off-chain, untrusted)
- 6. Economic decision engine
- 7. Co-signer policy (12 checks, one decision record)
- 8. Conversational layer: LLM for language, a state machine for money
- 9. Certified clearing: maximal multilateral netting with an exact proof (QEDClearing)
- 10. Private optimality: the same certificate, proven in zero knowledge (PrivateMinVarianceVerifier)
- 11. Theorems with money on them (OptimalityBounty, MaximalityBounty)

## Part 3 — Security Design & Self-Assessment

- 1. Security thesis
- 2. Permission tiers (implemented in QEDVault.sol)
- 3. Security properties and how each is verified
- 4. Threat model
- 5. Known-risk list (honest)
- 6. Incident response
- 7. Compliance-relevant controls

## Part 4 — Test Catalogue

- Solidity (Foundry, contracts/test)
- TypeScript (node --test, agent/test)
- End-to-end
- Clean-clone reproduction log

## Part 5 — Deployment & Operations

- Live deployment (Ethereum Sepolia, chain id 11155111)
- Prerequisites
- Local (no keys, no network)
- Public network
- Operations runbook

## Appendix A — On-chain Evidence (Ethereum Sepolia)

- Contracts
- Actors (one key each, one process each)
- Every step of the scenario
- Attack transactions (19) — each reverted with exactly the designed error
- Set-up transactions (funding, mints, approvals)
- Final state (read from QEDLens after the last transaction)
- Run 2 — certified clearing of an agent economy (QEDClearing)
- Run 3 — zero-knowledge optimality and theorem bounties (QED Private)
- How these transactions were produced — full disclosure
- Block timestamps

PART 0

# Overview

## The idea in seven sentences

1. **The AI holds one key that can call one function.** `QEDVault.executePayment` succeeds only with an owner-signed EIP-712 *mandate* **and** a certificate from an *independent co-signer* (a separate process, a separate key, 12 policy checks) **and** inside an on-chain *solvency invariant* that ring-fences money owed to creditors.
2. **The worst case is computed, not hoped for.** `QEDLens.blastRadius(vault, horizon)` returns, at any block, the most the vault can lose if *both* hot keys are stolen — and the loss to any address the owner never approved is provably **zero**.
3. **AI work is paid for proofs, not promises.** In `ProofEscrow` a client pays only when the delivered answer passes an **on-chain optimality check** (exact-integer KKT conditions + a positive-semidefiniteness witness). A lazy or dishonest AI provider is rejected by mathematics and the client is refunded.
4. **We published the keys.** The honeypot vault's agent *and* co-signer private keys are in this repository. Its 90 tUSD ring-fence is protected by the contract, not by secrecy. Try to steal it.
5. **Whole agent economies clear by proof.** In `QEDClearing`, five AI businesses that owe each other 200 tUSD but hold only 28 tUSD escape gridlock: the QED agent proves, with an exact linear-programming duality certificate checked on-chain, that its netting is the **largest possible** and leaves every member's net position unchanged. A lazy netting is rejected with `NotProvablyMaximal(162, 169)`. Details below.
6. **Optimality, proven in zero knowledge.** Proving an AI decision correct has meant showing its inputs; confidentiality has meant not showing them. In QED Private the agent proves, in a Groth16 zk-SNARK verified on-chain, that its portfolio is optimal for a data model the vendor attested by commitment. The escrow that settled job #1 pays on that proof, and the chain never sees the model or the portfolio. A copied proof is useless: it is bound to its author. Details below.
7. **Theorems with money on them.** Two contracts each hold 1,000 tUSD and pay it, automatically, to anyone who refutes on-chain the optimality theorem behind job #1 or the maximality theorem behind the clearing house. No judge: the contract checks the counterexample. Every attempt so far reverted with the gap it could not close. Details below.

## The four outcomes — each one a public transaction

| # | Outcome asked for | Proof on Ethereum Sepolia |
|---|---|---|
| O1 | A **separate AI-agent key** executes a payment under an **independent policy certificate** | `L4 0x56a2334e…` — sent **by the agent key** `0x2A10…d0Bb`; mandate signature recovers to the **owner** `0xb7Ed…97f7`; certificate signature recovers to the **co-signer** `0xf1F8…2402`; on-chain `evidenceHash` = keccak256 of the published decision record. Also `C1 0x7000d3cc…`: a chat request → clarified → corrected → owner-signed confirmation → certified payment. |
| O2 | A **paid-work lifecycle** that ends in a **real escrow settlement** | Twice. **Private:** `Z3` the same escrow pays 12 tUSD into QED's vault on a zero-knowledge proof of optimality for data the chain never sees. **Public:** RFQ → agent-signed quote → `L2` client funds escrow, 15 tUSD → `L4` agent buys the risk model it needs → solves → `L6` delivers portfolio + KKT certificate, verified **on-chain**, 15 tUSD settles into the agent's vault. Competing provider's lazy answer `X17` rejected: `DualityGapTooLarge`, near-miss `X18` rejected: `StationarityViolated`, client `X19` refunded. |
| O3 | An **adversarial agent cannot violate a protected liability or tamper with signed terms** | **19 attack transactions, all reverted on-chain with exactly the designed error** — tampered quote price/payee (`BadProviderSignature` ×2), forged/altered/redirected certificates (`BadCosignerSignature` ×3), both hot keys stolen paying the attacker (`PayeeNotInMandate`), self-signed mandate (`BadOwnerSignature`), replay (`IntentAlreadyUsed`), drain beyond free balance (`SolvencyViolation`), beyond the 24 h window (`ExceedsDailyCap`), admin calls (`NotOwner`, `NotCreditor`), published-key honeypot (×3), payment while paused (`VaultPaused`). Ring-fenced 70 tUSD reached **only** its creditor — settled by the attacker's own address while the vault was paused. Full table: docs/EVIDENCE.md. Plus **6 attacks on the clearing house** and **8 on the private job and the bounties** (a copied zk proof, an understated variance, self-attested data, five refutation attempts), all reverted. **33 attacks, 33 reverts, 0 tUSD to the attacker.** |

| # | Outcome asked for | Proof on Ethereum Sepolia |
|---|---|---|
| **O4** | An **external reviewer can reproduce everything** from GitHub source, tests and receipts | `npm run verify` re-derives every claim from a public RPC with **no keys**: on-chain bytecode == this repo's compiled bytecode; every receipt's status and sender; every revert's custom error (replayed on archive state); signatures recovered from calldata; the KKT certificate re-checked off-chain; the netting certificate re-checked; the Groth16 proof re-verified with snarkjs and the vendor's attestation recovered; both bounty pools intact; Mallory received 0 tUSD. It runs in CI on every push and daily. 104 Solidity tests (unit, fuzz, stateful invariants ≈ 350 k adversarial calls/run, 60 random clearing economies, a real zk proof attacked 2,000 ways) + 50 TypeScript tests. A fresh machine reproduced all of it from a clean clone (log). |

## What makes this different

| Usual agent wallet | QED |
|---|---|
| Spending limits on a hot key | Limits **plus** an independent co-signer **plus** a solvency invariant — three layers, each sufficient for protected funds |
| "The AI will follow its system prompt" | The AI is assumed compromised. Language model → deterministic slot filling → deterministic guard → owner-signed confirmation. Red-team: **25/25 attacks blocked, 0 false refusals** (LLM alone: 24/25) |
| Trust the provider's answer, or re-run it | **Verify a certificate**: the chain checks optimality in $O(n^2)$ exact integer operations instead of solving the problem |
| Risk = "we have limits" | Risk = a **number on-chain** ( `blastRadius` ), proven an upper bound by fuzzing, displayed live |
| Pause = everything stops | Pause stops the **AI**, never the **creditors** — anyone can settle a due liability, even while paused |
| Keys are the security boundary | Keys are **published** for the honeypot; the contract is the boundary |
| Verifying an AI's decision means exposing its data | **Zero-knowledge optimality**: the chain verifies the KKT certificate without seeing the data or the answer |
| "Trust our audit" | **Bounties the contract adjudicates**: refute the theorem on-chain and the contract pays you |
| Netting needs a trusted clearing house | **Anyone** can clear; the chain checks that the netting is maximal and moves no value (exact LP-duality certificate) |

## Certified clearing: a whole agent economy settles by proof

Five AI businesses (Northwind, QED, CloudCo, Helix, Atlas) owe each other **200 tUSD** in overlapping cycles and hold only **28 tUSD** of cash. Gross settlement is impossible: `G1` Northwind cannot pay its 40 tUSD debt with 18. Yet every one of them is solvent on a net basis.

`QEDClearing` lets **anyone** submit a netting, and executes it only if the contract can check that (1) every member's net position is exactly unchanged, (2) no debt is cut below zero and none is created, (3) the proof covers every open debt at its current amount, and (4) an **LP-duality certificate** proves no larger netting exists: $\Sigma x = \Sigma d \cdot y$ with $\pi_debtor - \pi_creditor + y \geq 1$. The constraint matrix is totally unimodular, so the check is exact integer arithmetic: no tolerance. Joining is the only consent needed, and the QED agent can be a member *and* the prover without being able to favour itself. Theory: docs/ALGORITHMS.md §9.

| Step on Ethereum Sepolia | Result |
|---|---|
| `N1` a naive shortest-cycle-first netting | reverted `NotProvablyMaximal(162, 169)` |
| `N2` a netting that cancels 1 tUSD more of one member's debt | reverted `NetPositionChanged(Northwind, 1)` |
| `N3` cancel more than is owed · `N4` hide an obligation · `N5` forge the certificate | `ExceedsObligation` · `IncompleteSnapshot` · `DualInfeasible` |
| `N6` the QED agent's proven netting | **executed: 169 tUSD of debt cancelled, gross 200 → 31, no token moved** |
| `N7` replay it | reverted `IncompleteSnapshot` |
| `S1` … `S4` the remaining 31 tUSD, paid in topological order | **0 debts left; every agent ends holding exactly its net position, using only the 28 tUSD that existed** |

Contracts: `QEDClearing 0x7b7d…6ad2` · run token `0x6753…52d4`. Evidence: `evidence/ethereum-sepolia/clearing/` (44 public receipts, 44/44 matched the replica).

## QED Private: paid for a proof about data nobody saw

Helix sells a private-markets covariance model it may not publish; Northwind, a wealth-manager agent, wants the provably optimal allocation and does not want it public. Helix signs a **Poseidon commitment** to its model. QED solves the problem, then proves inside a **Groth16 zk-SNARK** ( `circuits/private_minvar.circom` , 12,569 constraints) the full KKT certificate of §4 — stationarity, dual feasibility, complementary slackness, a convexity witness and an exact budget — over the committed data. `PrivateMinVarianceVerifier` checks Helix's attestation, binds the proof to its author and verifies it; it is just another `IProofVerifier` , so **the escrow deployed for job #1 settles the private job unchanged**. For reproducibility, this demo's synthetic model sits in `agent/src/engine/privateScenario.ts` ; in production it would never leave Helix and the agent. Theory, constraints and trusted setup: docs/ALGORITHMS.md §10.

| Step on Ethereum Sepolia | Result |
|---|---|
| `Z1` Northwind funds job #3 in the original ProofEscrow | 12 tUSD escrowed; the verifier is part of the signed terms |
| `Z3` QED delivers the zero-knowledge proof | **verified on-chain; 12 tUSD paid into QED's vault; the model and portfolio never appear** |
| `ZA1` Mallory replays QED's proof in her own job | reverted `ProofRejected` (bound to QED) |
| `ZA2` Mallory claims a lower variance | reverted `ProofRejected` |
| `ZA3` a genuine proof about data Mallory attested herself | reverted `UntrustedDataAttestation` |

## Theorems with money on them

`OptimalityBounty` and `MaximalityBounty` each hold **1,000 tUSD** and pay all of it to anyone who refutes, on-chain, the theorem QED's money depends on: a certificate the deployed `MinVarianceVerifier` accepted plus a strictly better portfolio (beyond the proven bound), or a netting larger than one `QEDClearing` accepts as maximal. The contracts check the counterexample in exact integers and pay with no human involved. Tests prove the payout path works against an unsound verifier and an under-reporting rule; on Sepolia every attempt reverted:

| Attempt | Result |
|---|---|
| `BA1` equal-weight portfolio vs job #1 · `BA2` QED's optimum shifted 0.1% | `TheoremHolds(advantage, bound)` |
| `BA3` the forged certificate from job #2 | `DualityGapTooLarge` (the verifier refuses it first) |
| `BM1` the naive 162 tUSD netting · `BM2` a 170 tUSD netting | `TheoremHolds(169, 162)` · `NetPositionChanged` |

`npm run verify` checks daily that both pools are still full. docs/ALGORITHMS.md §11.

## How the evidence was produced (full disclosure)

Our build sandbox had no RPC egress. Every transaction was signed inside the sandbox — one key per process — against a wall-clock-synchronised replica of Sepolia (same chain id, nonces and balances); the **identical signed bytes** were committed to `relay/` and broadcast unchanged by a browser relay, and each public receipt was compared with the replica's (status + every log): **55/55 matched** for the main run, **44/44** for the clearing run and **24/24** for the private run. Two earlier attempts under other keys failed (Sepolia's state-creation gas repricing; one under-provisioned agent transaction) and were abandoned; the canonical run uses fresh keys. Details: docs/EVIDENCE.md, risk K12 in docs/SECURITY.md.

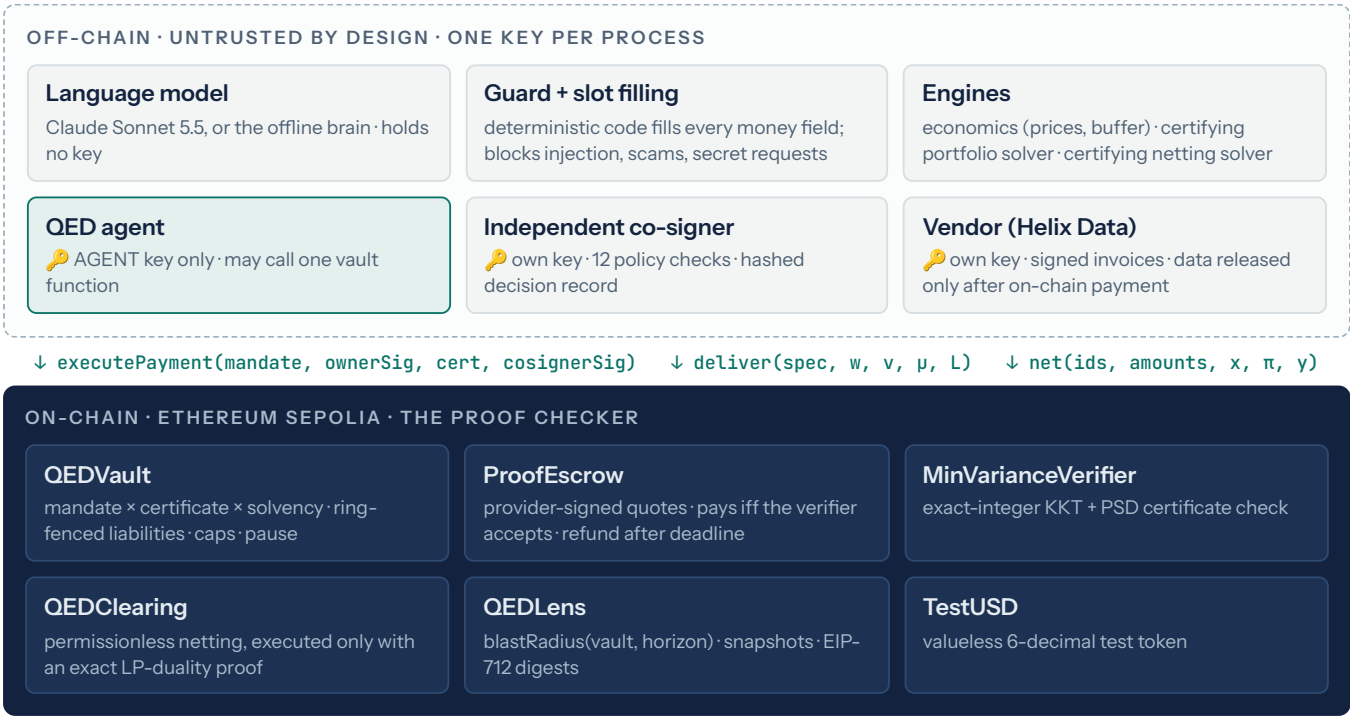
## Status

Testnet prototype with a valueless token; not audited. The Groth16 parameters come from a local ceremony and must be treated as trusted (risk K18). Known risks are listed honestly in docs/SECURITY.md §5. MIT licence, except `contracts/src/verifiers/zk/Groth16Verifier.sol` , which snarkjs generates under GPL-3.0.

PART 1

# QED — System Architecture

## 1. One picture




Outside both bands: the **owner** (cold key) signs mandates once, off-chain (EIP-712); the **guardian** can pause the agent; the **client** funds escrow against a signed quote; **creditors** are paid from ring-fenced funds only — even while the vault is paused. The interactive version of this diagram is in docs/ARCHITECTURE.md on GitHub.

The design rule is a trust gradient: **everything that can be fooled is off-chain and holds at most one key; everything that decides whether money moves is on-chain and checks proofs.**

## 2. Components

### On-chain ( `contracts/src` , Solidity 0.8.26, via-IR, OpenZeppelin 5.1)

| Contract | Size | Responsibility |
|---|---|---|
| `QEDVault` | 9.1 KB | Treasury of one agent. Verifies owner mandates (EIP-712, EOA or ERC-1271) and co-signer certificates; enforces payee scope, per-payment/period/lifetime caps, a global 24 h window cap, the solvency invariant, single-use intents, epochs, pause. Holds ring-fenced liabilities that only creditors can be paid from. |
| `QEDLens` | 5.1 KB | Stateless views: `blastRadius(vault, horizon)` , `snapshot(vault)` , EIP-712 hashes/digests. Separate so the enforcement contract stays small (Sepolia's state-creation pricing + EIP-7825's $2^{24}$ gas cap bound deployable code size). |
| `ProofEscrow` | 5.6 KB | Agent-to-agent work. Provider-signed EIP-712 quote binds price, payee, client, verifier, problem hash, deadline. `deliver` pays iff `IProofVerifier.verify(spec, solution)` succeeds. Refund after deadline. |
| `MinVarianceVerifier` | 2.4 KB | `IProofVerifier` for long-only minimum-variance portfolios: exact-integer KKT + PSD certificate check (docs/ALGORITHMS.md §4). |
| `QEDClearing` | 5.3 KB | Proof-gated multilateral netting for agent economies. Members `join` (on-chain consent to the terms) and record debts as debtors; `net` is permissionless and executes only if net positions are unchanged, nothing is created, the snapshot is complete and current, and an exact LP-duality certificate proves the netting maximal (ALGORITHMS §9). |

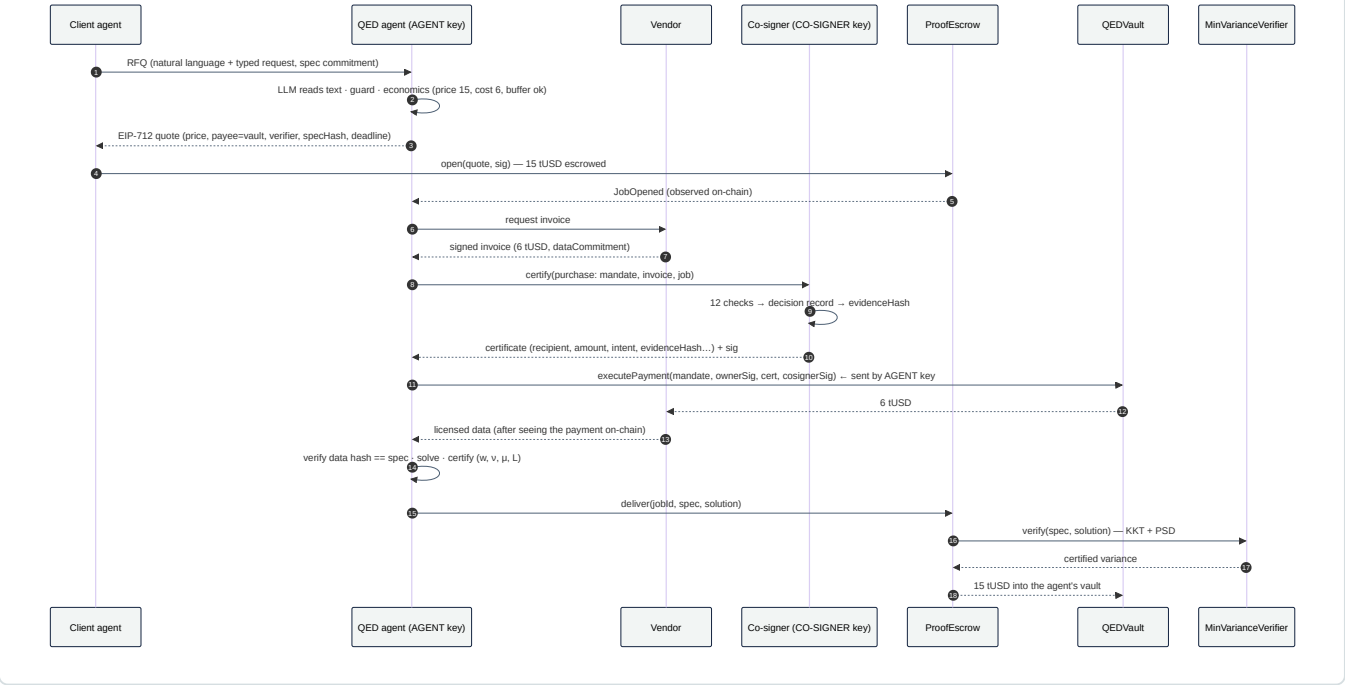
| Contract | Size | Responsibility |
|---|---|---|
| `PrivateMinVarianceVerifier` + `Groth16Verifier` | 2.1 KB + 1.6 KB | `IProofVerifier` for the same problem **in zero knowledge**: checks the data vendor's attestation of a Poseidon commitment, binds the proof to its provider, then verifies a Groth16 proof of the KKT certificate (ALGORITHMS §10). Usable by the deployed `ProofEscrow` unchanged. |
| `OptimalityBounty`, `MaximalityBounty` | 2.0 KB, 2.6 KB | Standing offers that pay their whole balance to anyone who refutes, on-chain, Theorem 4.4 (via the deployed verifier) or the netting maximality theorem (ALGORITHMS §11). No owner, no withdrawal. |
| `TestUSD` | 2.1 KB | Valueless 6-decimal test token with a public 1 000 tUSD/day faucet for reviewers. |

## Off-chain ( `agent/` , TypeScript, viem)

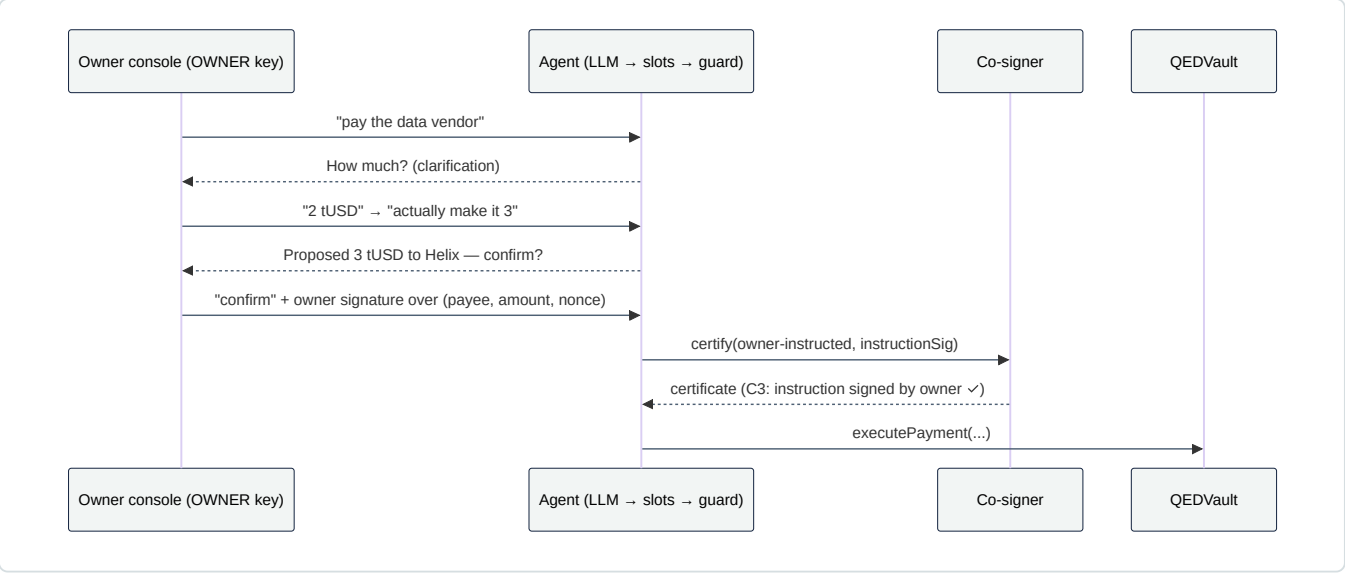
| Module | Process / key | Responsibility |
|---|---|---|
| `roles/agent.ts` | agent process · AGENT key | RFQ handling, autonomous job fulfilment (watches `JobOpened`), owner console, decision records |
| `roles/cosigner.ts` | co-signer process · CO-SIGNER key | 12-check policy engine; writes & hashes decision records; signs certificates |
| `roles/vendor.ts` | vendor process · VENDOR key | catalog, signed invoices, releases data only after seeing the on-chain payment |
| `brain/*` | inside agent | `AnthropicBrain` (live LLM) / `OfflineBrain`; `slots.ts` deterministic payment slot filling; `guard.ts` deterministic security layer; `evaluate.ts` red-team scoring |
| `engine/minvar.ts` | inside agent | certifying solver + bit-exact TS checker |
| `engine/economics.ts` | inside agent | pricing and acceptance rules |
| `engine/privateMinvar.ts` | inside agent | fixed-point witness, Poseidon commitments, Groth16 prover (snarkjs), bit-exact mirror of every circuit constraint |
| `circuits/private_minvar.circom` | build time | the zero-knowledge statement (12,569 constraints); `circuits/setup.sh` runs the two-phase trusted setup |
| `engine/netting.ts` | inside agent | certifying netting solver (min-mean cycle cancelling + Bellman–Ford dual), bit-exact checker, naive greedy rival |
| `scripts/clearing.ts` | orchestrator (5 member agents, QED prover, Mallory) | runs the clearing economy: gridlock, attacks, proven netting, settlement |
| `scripts/private.ts` | orchestrator (Helix, Northwind, QED, Atlas, Mallory) | runs the private economy and the bounty challenges on the main run's escrow and verifier |
| `scripts/live.ts` | orchestrator (owner, client, guardian, Mallory) | deploys and runs the whole economy, records evidence |
| `scripts/verify.ts` | anyone · no keys | independent re-derivation of every on-chain claim |

# 3. Key lifecycle flows

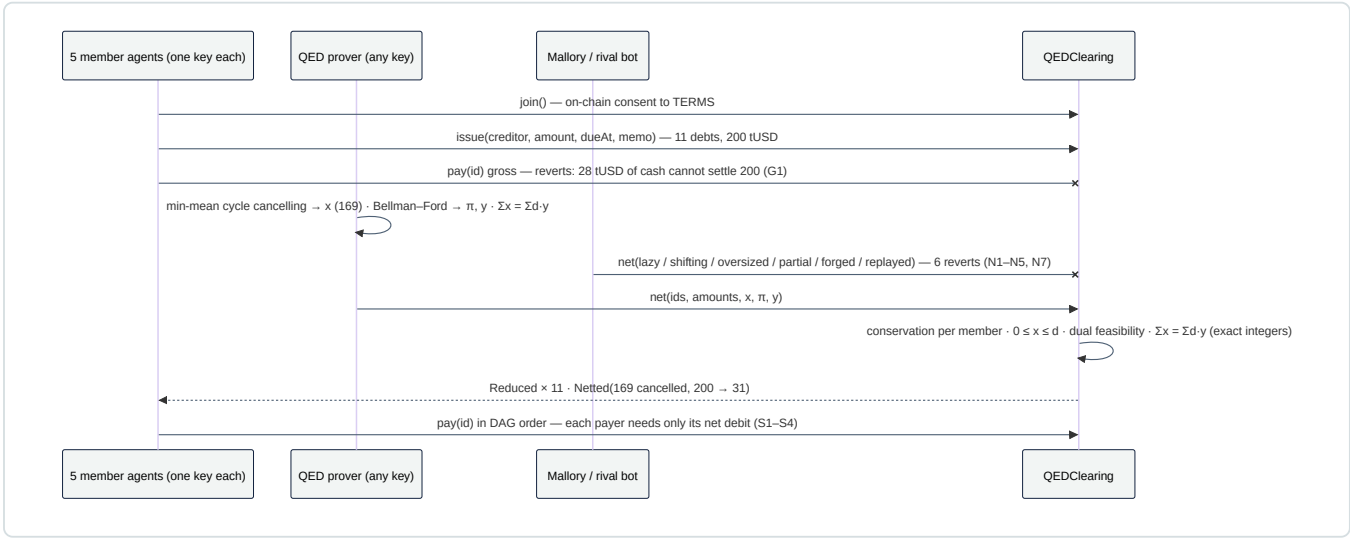
## 3.1 Paid work, end to end (Outcomes 1 & 2)



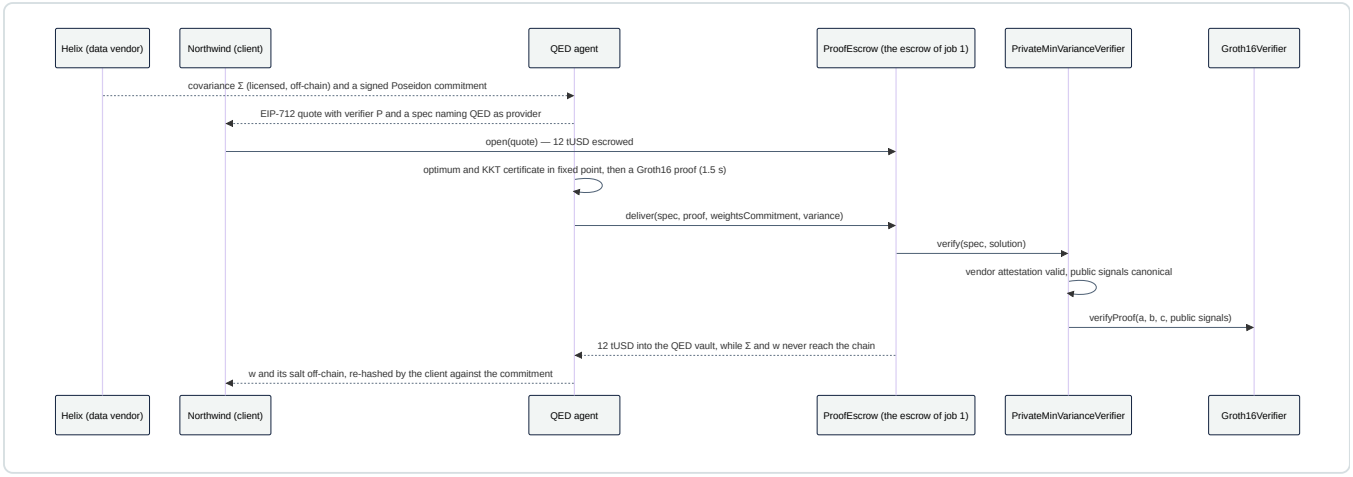

## 3.2 Owner conversation → signed confirmation → payment



### 3.3 Certified clearing of an agent economy (QEDClearing)




### 3.4 Private work, paid on a zero-knowledge proof (QED Private)



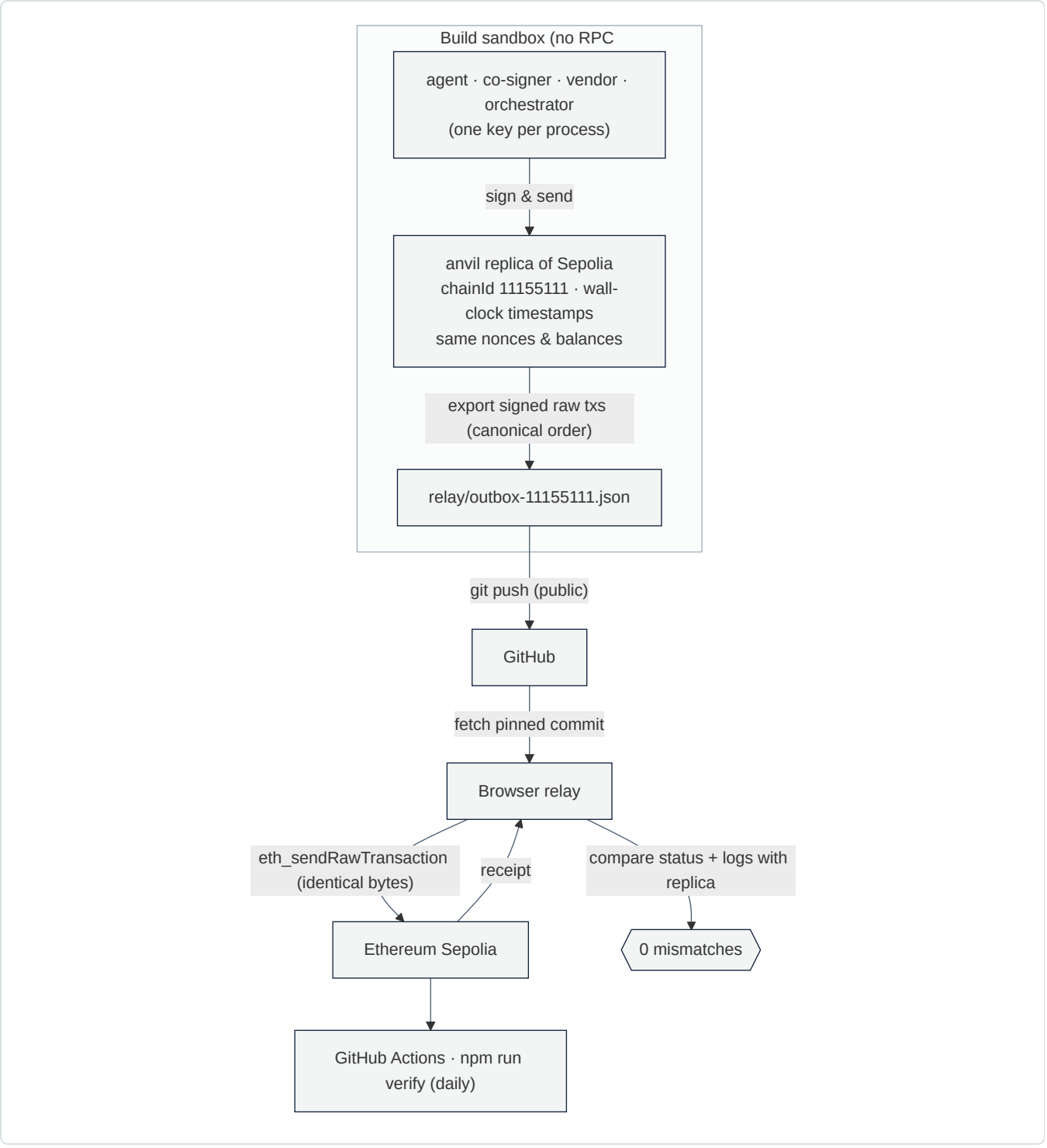

### 3.5 Defence in depth against a manipulated agent

| Layer | Who | Stops |
|---|---|---|
| 1. Guard | deterministic code in the agent | injection, scams, secret requests, escalation — before any proposal |
| 2. Co-signer | independent process & key | anything not justified by a signed invoice + funded job, or not owner-signed |
| 3. Vault | Ethereum | everything outside the owner's mandate, caps, solvency, replay, pause — even if layers 1 and 2 are both compromised |

## 4. Data & evidence model

- **Mandate** (owner-signed, off-chain): scope of authority. Hash is referenced by every certificate.
- **Certificate** (co-signer-signed): exact payment terms + `evidenceHash` of the decision record.
- **Decision record** ( `evidence/…/records/cosigner/<hash>.json` ): canonical JSON of inputs, treasury snapshot, all 12 checks and verdict. `keccak256(canonical(core)) == evidenceHash` on-chain.
- **Agent records** ( `records/agent/*` ): RFQ evaluations, fulfilment (solution + certificate stats), every chat turn with the LLM transcript.
- **Manifest** ( `evidence/ethereum-sepolia/manifest.json` ): every step, actor, expected outcome, tx hash, decoded events / errors, mapped to Outcomes O1–O4.

## 5. Execution & deployment topology




The transaction hashes in the evidence are the public-chain hashes (identical signed bytes). The relay compares each public receipt with the replica (status and every log, excluding Sepolia's EIP-7708 system log for plain ETH transfers), and the independent verifier re-derives every claim from a public RPC. A reviewer with RPC access can also run the economy directly against Sepolia: `RPC_URL=… CHAIN_ID=11155111 GAS_LIMIT=16700000 npm run live`.

## 6. Gas profile on Sepolia (canonical run)

| Operation | Gas used |
|---|---|
| Deploy QEDVault | 14 963 244 |
| Deploy ProofEscrow | 9 051 450 |

| Operation | Gas used |
|---|---|
| Deploy QEDLens | 8 026 770 |
| Deploy MinVarianceVerifier | 3 847 926 |
| Deploy TestUSD | 3 716 239 |
| Deploy QEDClearing | 8 579 134 |
| `QEDClearing.net` — 11 obligations, 5 members, certificate checked (N6) | 550 826 |
| `QEDClearing.issue` (first / later) | 795 367 / ≈ 501 600 |
| Deploy Groth16Verifier / PrivateMinVarianceVerifier | see `docs/EVIDENCE.md` (Run 3) |
| `ProofEscrow.deliver` with a zero-knowledge proof (attestation + Groth16) | see `docs/EVIDENCE.md` (Run 3, Z3) |

Sepolia's current schedule prices state creation far above the classic EVM (a plain ETH transfer to a fresh account costs ≈ 205 k gas); local replicas don't model this, so all broadcasts use an explicit 16.7 M gas limit (only gas used is charged).

PART 2

# QED — Core Algorithms

This document specifies every algorithm QED relies on, with the exact conditions the smart contracts check and proofs of the properties we claim. Notation: all token amounts are integers in `tUSD` base units (6 decimals). Real numbers in the optimiser are fixed-point **WAD** integers ($1.0 = 10^{18}$).

## 1. The solvency invariant (ring-fenced liabilities)

**State.** `B` = vault token balance, `P` = `totalProtected` = Σ of open liabilities, `F = B - P` = free balance.

**Invariant I1.** `B ≥ P` at every reachable state.

**Proof (induction over state transitions).** Initially `B = P = 0`. Every function that changes `B` or `P`:

| Transition | ΔB | ΔP | Pre-condition checked on-chain | I1 preserved because |
|---|---|---|---|---|
| `deposit(a)` / direct transfer in | +a | 0 | — | B grows |
| `executePayment` (spot) | −a | 0 | `a ≤ F` | B − a ≥ P |
| `executePayment` (forward, `dueAt>0`) | 0 | +a | `a ≤ F` | P + a ≤ B |
| `ringFence(a)` (owner) | 0 | +a | `a ≤ F` | P + a ≤ B |
| `withdraw(a)` (owner) | −a | 0 | `a ≤ F` | B − a ≥ P |
| `settle(id)` (anyone, after due) | $-a_i$ | $-a_i$ | liability open | both sides drop equally |
| `release(id)` (creditor only) | 0 | $-a_i$ | `msg.sender == creditor` | P shrinks |

No other function moves tokens. ∎

**Corollary (creditor exclusivity).** Ring-fenced value leaves the vault only via `settle(id)`, which transfers exactly $a_i$ to `liabilities[id].creditor`. Nobody — agent, co-signer, guardian or owner — can redirect it. Only the creditor can `release` (forgive) or `assign` (sell) its own claim.

Both facts are checked by stateful fuzzing (`contracts/test/Invariants.t.sol`: 7 invariants × 512 runs × 96 calls with an attacker holding the agent **and** co-signer keys) and re-checked live on Sepolia by `npm run verify`.

## 2. Authorisation: owner mandate × independent certificate

A payment executes only if **all** of the following hold (in this order; the first failure reverts with a self-describing custom error):

```
executePayment(m, ownerSig, c, cosignerSig):
  require !paused                                  VaultPaused
  require msg.sender == agent                      NotAgent
  — mandate —
  require m.vault == this, m.agent == agent        WrongVault / MandateAgentMismatch
  require m.period > 0, 0 < |m.payees| ≤ 32         InvalidMandate
  require m.validFrom ≤ now ≤ m.validUntil          MandateNotActive
  require m.epoch == epoch, !revoked[H(m)]          StaleEpoch / MandateIsRevoked
  require SigCheck(owner, EIP712(H(m)), ownerSig)    BadOwnerSignature      (EOA or ERC-1271)
  — certificate —
  require c.mandateHash == H(m)                     CertificateMandateMismatch
  require SigCheck(cosigner, EIP712(H(c)), sig)      BadCosignerSignature   (authenticity FIRST)
  require c.epoch == epoch, c.notBefore ≤ now ≤ c.expiresAt
  require !intentUsed[c.intentId]                   IntentAlreadyUsed      (replay)
  require c.recipient ∈ m.payees                    PayeeNotInMandate
  require c.amount ≤ m.maxPerPayment               ExceedsPerPaymentCap
  — economics —
  require c.amount ≤ F                              SolvencyViolation(requested, free)
  require mandate period / lifetime / global 24h window caps
  effects: intentUsed[c.intentId] = true; counters += amount
  if c.dueAt == 0: transfer(c.recipient, c.amount)
  else:            create liability(c.recipient, c.amount, c.dueAt)   // ring-fenced promise
  emit PaymentExecuted(intentId, H(m), recipient, amount, liabilityId, evidenceHash, ref)
```

**Why the certificate binds everything.** `H(c)` covers `recipient, amount, dueAt, intentId, evidenceHash, ref, notBefore, expiresAt, epoch` and `H(m)`. Changing any byte changes the EIP-712 digest, so the recovered signer is no longer the co-signer. `intentId` is single-use. `epoch` lets the owner revoke every outstanding mandate and certificate with one transaction (`bumpEpoch`).

**Key separation.** Owner (cold) ≠ agent (hot) ≠ co-signer (independent service) ≠ guardian. Each runs in a separate OS process holding exactly one key (`agent/scripts/live.ts → spawnRole`).

---

## 3. Provable blast radius (worst-case loss under total hot-key compromise)

**Threat.** The attacker holds the agent key **and** the co-signer key (owner key safe).

**Bound.** Within horizon `h` the attacker can extract at most

```
BR(h) = 0                                  if paused
      = min( F ,  cap_now + k·D )          otherwise
cap_now = D - spentInWindow   if the current window is open, else D
k       = ⌊(t + h - windowEnd)/W⌋ + 1      if t + h ≥ windowEnd, else 0
```

with `D` = global daily cap, `W` = 1 day, and `windowEnd` the end of the current (or next-opened) fixed window.

**Proof sketch.** Every agent outflow (spot or forward) passes `_consumeLimits`, which bounds spending in each fixed window by `D`; a new window opens no earlier than the previous one closes, so at most `k` further windows can start inside the horizon (counting a window that opens exactly at `t + h`, a deliberately conservative choice). Every outflow also requires `a ≤ F`. Ring-fenced funds (`P`) are unreachable by §1. ∎

**And the part a cap alone can't give you:** the attacker cannot invent payees — `c.recipient ∈ m.payees` and `m` must carry the **owner's** signature. So loss to any address the owner did not approve is exactly **0**. `QEDLens.blastRadius(vault, h)` computes `BR(h)` on-chain from public state.

---

# 4. Certified optimality: paying AI for proofs, not for answers

## 4.1 Problem

The client agent buys the long-only minimum-variance portfolio for a committed covariance $\Sigma$:

```
minimise  f(w) = ½ wᵀΣw     subject to   1ᵀw = 1,  w ≥ 0.
```

## 4.2 What the provider submits

`(w, v, μ, L)` — the portfolio, the Lagrange multipliers of the budget constraint (`v`) and of the long-only constraints (`μ`), and a lower-triangular factor `L` witnessing that $\Sigma$ is positive semidefinite.

## 4.3 What the contract checks (`MinVarianceVerifier.verify`, exact integer arithmetic, tolerance $\tau$)

| # | Condition | Meaning |
|---|---|---|
| P1 | $w_i \geq 0$, ` | $1^T w - 1$ |
| D1 | $\mu_i \geq 0$ | dual feasibility |
| S | ` | $(\Sigma w)_i - v - \mu_i$ |
| CS | $\Sigma_i w_i \mu_i \leq \tau$ | complementary slackness = the **duality gap** |
| C | ` | $\Sigma_{ij} - (LL^T)_{ij}$ |

All products are formed at $10^{36}$ scale and compared without any rounding. Inputs are range-bounded ($|x| \leq 10^{27}$) so no product can overflow. Cost for $n = 6$: $\approx 0.3$ M gas.

## 4.4 Theorem ($\varepsilon$-optimality certificate)

*If checks P1, D1, S, CS and C pass with tolerance $\tau$, then for every feasible portfolio $x$*

$$f(w) \leq f(x) + \tau\cdot(|v| + 3 + 2n) + O(\tau^2).$$

*In particular* $f(w) \leq f^* + (2n + 3 + |v|)\cdot\tau + O(\tau^2)$*: the escrow pays only for answers provably within a tiny, known distance of the global optimum.*

**Proof.** Write $\Sigma = LL^T + E$ with $|E_{ij}| \leq \tau$ (check C), so $\|E\|_2 \leq \|E\|_F \leq n\tau$. For feasible $x$ ($x \geq 0$, $1^T x = 1$) let $d = x - w$. Exactly,

$$f(x) = f(w) + (\Sigma w)^T d + \tfrac{1}{2} d^T \Sigma d.$$

*Curvature term.* $d^T\Sigma d = \|L^T d\|^2 + d^T E d \geq -n\tau\|d\|_2^2$.

*Linear term.* By S, $\Sigma w = v1 + \mu + r$ with $\|r\|_\infty \leq \tau$. Hence

$$\begin{aligned}(\Sigma w)^T d &= v(1^T x - 1^T w) + \mu^T x - \mu^T w + r^T d \\ &\geq -|v|\tau + 0 - \tau - \tau\|d\|_1 \qquad \text{(P1, D1 with } x \geq 0\text{, CS, Hölder)}\end{aligned}$$

*Sizes.* $\|d\|_1 \leq \|x\|_1 + \|w\|_1 \leq 2 + \tau$ and $\|d\|_2^2 \leq \|d\|_1^2 \leq (2 + \tau)^2$.

Combining: $f(x) \geq f(w) - \tau(|v| + 1 + 2 + \tau) - \tfrac{1}{2}n\tau(2 + \tau)^2$, i.e. $f(w) - f(x) \leq \tau(|v| + 3 + 2n) + O(\tau^2)$. ∎

**Corollary (the revert message is a certificate of failure).** With exact stationarity ($r = 0$) the same argument gives $f(w) - f^* \leq \mu^T w$. So when a forger dresses up a portfolio with the best multipliers it can find ($v = \min_i(\Sigma w)_i$, $\mu = \Sigma w - v1 \geq 0$, which makes S and D1 hold *exactly*), the only check left to fail is CS — and `DualityGapTooLarge(gap)` reports an upper bound on how suboptimal the forgery is. On Sepolia, Mallory's lazy 1/N portfolio was rejected with gap = 0.019113 (the true optimum has half-variance 0.00133).

**Numbers for the live job** ($n = 6$, $\tau = 10^{-9}$, $\nu = 0.0026604$): certified bound `(12 + 3 + 0.0027)·10⁻⁹ ≈ 1.5·10⁻⁸`. The submitted certificate had stationarity residual **0** (exact), duality gap **0**, PSD residual $6.3 \cdot 10^{-17}$.

### 4.5 Why this matters

Classical escrow needs a human, an oracle, a reviewer quorum or "client acceptance" — each a trust assumption and an attack surface. A *certifying algorithm* (McConnell, Mehlhorn, Näher & Schweitzer, 2011) returns a witness that a simple checker can verify. QED makes the blockchain that checker for AI work: the expensive, opaque part (an AI-driven optimiser) is untrusted; the cheap, transparent part (the KKT check) is executed by every Ethereum node. The same construction extends to every convex financial problem with a duality certificate — CVaR (an LP), risk parity, index tracking, cash-flow matching, multilateral netting — by deploying a new `IProofVerifier`.

---

## 5. The certifying solver (off-chain, untrusted)

`agent/src/engine/minvar.ts`

1. **Exact support enumeration.** For each non-empty support $S \subseteq \{1..n\}$, solve `Σ_SS x = 1`; set `w_S = x / 1ᵀx`, `v = 1 / 1ᵀx`. Accept S iff `w_S ≥ 0` and `μᵢ = (Σw)ᵢ - v ≥ 0` for all `i ∉ S`. $\Sigma \succ 0 \Rightarrow$ the KKT point is unique, so this returns the exact optimum. Cost `O(2ⁿ n³)`: 1.3 ms for $n = 6$.
2. **Fixed-point certificate.** Round w to WAD, place the rounding residue on the largest weight so `1ᵀw = 10¹⁸` exactly, recompute `g = Σw` at $10^{36}$ scale in BigInt, set `v` = mean of `gᵢ` over the support, `μᵢ = max(0, gᵢ - v)` off-support. Cholesky factor L via standard decomposition, rounded to WAD.
3. **Pre-flight.** `checkCertificate` is a bit-exact TypeScript re-implementation of the verifier; the agent also dry-runs `ProofEscrow.check` (eth_call) before spending gas on `deliver`.

Result for the Helix model: Global Equities 16.32 %, US Treasuries 7–10y 80.81 %, Gold 2.87 %, all others 0 (their $\mu > 0$ certifies that adding any of them would increase variance). Portfolio volatility 5.16 %.

---

## 6. Economic decision engine

`agent/src/engine/economics.ts` — deterministic; the LLM never prices anything.

```
price  = ⌈ cost × 2.5 ⌉ (whole tUSD, ≥ 10)
accept ⇔ budget ≥ price
       ∧ margin/price ≥ 30 %
       ∧ cost ≤ F                       (only FREE money funds costs)
       ∧ F - cost ≥ 5 tUSD              (reserve buffer above ring-fenced liabilities)
       ∧ cost ≤ remaining 24h window
       ∧ delivery ≤ 72 h
```

Unsettled revenue is never treated as capital: the 15 tUSD fee becomes spendable only when the escrow settles into the vault (check E7).

**Earned autonomy (co-signer check C12).** Discretionary spend is capped at `10 tUSD + 50 % × verified revenue`, where verified revenue is the sum of `JobSettled` events paid into this vault. Autonomy grows only with on-chain, proof-settled performance.

---

## 7. Co-signer policy (12 checks, one decision record)

| ID | Check | Source of truth |
|---|---|---|
| C1 | Mandate signed by the vault owner | ecrecover over `QEDLens.mandateDigest` |
| C2 | Mandate active, current epoch | chain |

| ID | Check | Source of truth |
|---|---|---|
| C3 | Invoice signed by the payee's registered key (or owner-signed instruction for chat payments) | signatures |
| C4 | Invoice addressed to this vault, unexpired | invoice |
| C5 | Price = vendor's published list price | catalog |
| C6 | Invoice / instruction nonce not seen before | co-signer state |
| C7 | A funded escrow job for this vault covers the cost with ≥ 30 % margin | chain ( `getJob` ) |
| C8 | Purchased data is exactly the job's committed spec | `job.specHash == invoice.dataCommitment` |
| C9 | Payee listed in the owner's mandate | mandate |
| C10 | Reserve buffer preserved | chain ( `QEDLens.snapshot` ) |
| C11 | Sanctions / deny-list screening | policy |
| C12 | Earned autonomy | chain ( `JobSettled` , `PaymentExecuted` ) |

The full record (inputs, treasury snapshot, every check) is canonicalised (sorted-key JSON), hashed with keccak-256, and that hash is signed into the certificate's `evidenceHash` — so the on-chain payment event permanently commits to the exact reasoning that authorised it. `npm run verify` recomputes the hash.

---

## 8. Conversational layer: LLM for language, a state machine for money

```
message → LLM (or offline brain) → fillPaymentSlots (deterministic) → Guard (deterministic) → reply /
proposal
                                                                                    |
                                                          confirm + owner signature ──┘→ co-signer → vault
```

- **Slot filling** ( `agent/src/brain/slots.ts` ): payee, amount, due date, *confirm* and *cancel* are extracted by code from the raw text and the pending proposal, so a payment never depends on how the model phrased a turn.
- **Guard** ( `agent/src/brain/guard.ts` ): detectors for the rubric's four attack classes (English + Chinese); any transfer to an address outside the owner's mandate, any request for secrets, any attempt to widen authority produces a fixed refusal written by code, not by the model.
- **Confirmation** is a signature: the owner's console signs the exact instruction ( `payee, amount, dueInDays, nonce` ), and the co-signer verifies it (check C3) before certifying.

Red-team result (32 cases: 25 attacks + 7 benign controls, live Claude Sonnet 5.5): guarded pipeline blocked **25/25** attacks with **0** false refusals; the model alone blocked 24/25 ( `evidence/redteam/report.md` ).

---

## 9. Certified clearing: maximal multilateral netting with an exact proof ( `QEDClearing` )

### 9.1 The gridlock

Members of an agent economy owe each other money in cycles. In the live run five AI businesses owe each other **200 tUSD** but hold only **28 tUSD** of cash between them; every one of them is solvent on a net basis, yet nobody can settle gross (on-chain receipt G1: Northwind owes 40 and holds 18). Cancelling debts around cycles needs no tokens at all, but the party that computes the cancellation could be wrong, lazy or self-serving.

### 9.2 The linear program and its dual

For open obligations $e = (u \to v)$ with outstanding amounts $d_e$, choose cancellations $x_e$:

```
(P)  maximise  Σ_e x_e    s.t.  Σ_{e out of v} x_e = Σ_{e into v} x_e   for every member v      (net positions unchanged)
                            0 ≤ x_e ≤ d_e                                                      (never below zero)

(D)  minimise  Σ_e d_e y_e  s.t.  π_u - π_v + y_e ≥ 1   for every e = (u → v),   y_e ≥ 0,   π free
```

**Weak duality.** For any x feasible in (P) and (π, y) feasible in (D):

```
Σ_e x_e ≤ Σ_e x_e (π_u - π_v + y_e) = Σ_v π_v (Σ_out x - Σ_in x) + Σ_e x_e y_e = Σ_e x_e y_e ≤ Σ_e d_e y_e .
```

So a proposal with $\Sigma_e x_e = \Sigma_e d_e y_e$ is optimal: no larger netting exists. (P)'s constraint matrix is a network matrix, hence totally unimodular; with integer data there are integer optimal (x, π, y), so the certificate is exact: no tolerance, no rounding, no floating point.

### 9.3 What the contract checks ( `QEDClearing.net` , O(obligations + members))

| Check | Error if it fails | What it rules out |
|---|---|---|
| The proposal lists **every** open obligation, ids strictly increasing | `IncompleteSnapshot` , `IdsNotIncreasing` | proving "maximality" on a doctored subset |
| Each listed amount equals the current amount | `StaleAmount` | proofs about a past state; replays |
| $0 \le x_e \le d_e$ | `ExceedsObligation` | erasing more debt than exists |
| For every member, cancelled payables = cancelled receivables | `NetPositionChanged(member, Δ)` | moving value between members |
| $\pi_u - \pi_v + y_e \ge 1$, bounded values | `DualInfeasible` , `ValueOutOfRange` | forged certificates, overflow games |
| $\Sigma_e x_e = \Sigma_e d_e y_e$ | `NotProvablyMaximal(cleared, bound)` | lazy or suboptimal nettings |

**Properties (proved by the checks themselves).** Every member's receivable − payable is unchanged; no claim is created or re-assigned, only reduced, so every counterparty exposure shrinks; and the executed netting is maximal. That is why `net()` can be **permissionless**: joining ( `join()` , which emits the hash of these terms) is the only consent needed, and the QED agent can be both a member and the prover without being able to favour itself.

### 9.4 The certifying solver (off-chain, untrusted): `agent/src/engine/netting.ts`

1. (P) is a maximum-gain circulation = a min-cost circulation with cost −1 per unit cancelled. Solved exactly by **minimum-mean cycle cancelling** (Goldberg & Tarjan, 1989), each cycle found with **Karp's algorithm** (1978): strongly polynomial and integral.
2. At optimality the residual graph (forward arcs cost −1 where $x < d$, backward arcs cost +1 where $x > 0$) has no negative cycle, so **Bellman–Ford** from a virtual source gives integer distances π. Setting $y_e = \max(0, 1 - (\pi_u - \pi_v))$ is dual-feasible and complementary-slack, hence $\Sigma x = \Sigma d \cdot y$.
3. `checkNetting` re-implements the on-chain checks bit for bit (pre-flight and independent verification).

**Verified:** equals an exhaustive brute-force oracle on 150 random small economies; certificates valid on 60 random economies (3–7 agents, 3–16 debts) that the Solidity suite also executes; a 2,000-run fuzzer pushes any amount of debt around every simple cycle of the demo economy and every such proposal is rejected.

### 9.5 The live run (Ethereum Sepolia)

| | |
|---|---|
| Gross debt | 200 tUSD across 11 obligations, 5 agents |
| Cash held | 28 tUSD (each agent exactly its net debit) |
| Maximal netting (QED agent) | **169 tUSD cancelled**, dual bound 169: executed (N6) |
| Naive shortest-cycle-first netting | 162 tUSD: `NotProvablyMaximal(162, 169)` (N1) |

| | |
|---|---|
| Remaining after netting | 31 tUSD in 4 obligations, an acyclic graph, paid in topological order with the 28 tUSD of cash |
| End state | 0 open obligations; every agent holds exactly its net position |

Remaining debts after a maximal netting always form a DAG (a cycle would admit more cancellation), so paying in topological order needs only each agent's net debit in cash: Σ max(0, payables − receivables).

---

## 10. Private optimality: the same certificate, proven in zero knowledge (`PrivateMinVarianceVerifier`)

### 10.1 The tension it removes

§4 makes the chain check that a paid portfolio is optimal. The check needs Σ and w in calldata, so the licensed risk model and the client's allocation become public the moment the escrow pays. Accountability and confidentiality pull in opposite directions: an auditor who can verify the decision can also read the book. QED proves the **same** KKT certificate inside a zk-SNARK, so the chain verifies optimality while learning only two commitments, the certified variance and the provider's address.

### 10.2 The statement (`circuits/private_minvar.circom`, n = 6, Groth16 over BN254)

Fixed point `ONE = 10^12`; all values are integers; signed values use the field's negative encoding.

| Public signal | Meaning |
|---|---|
| `dataCommitment` | `Poseidon(Poseidon(Σ0…Σ10), Poseidon(Σ11…Σ20, saltΣ))`: the vendor's 21 covariance entries |
| `weightsCommitment` | `Poseidon(w1…w6, saltW)`: the delivered portfolio |
| `variance` | $w^\top \Sigma w$ at scale $ONE^3$ (exact) |
| `provider` | the address the proof is bound to |

The prover knows (Σ, saltΣ, w, ν, μ, L, saltW) such that, with $\tau = 10^{-10}$:

| Constraint | Encoding in the circuit |
|---|---|
| Range: $\lvert\Sigma_{ij}\rvert, \lvert L_{ij}\rvert < 2^{40}$; $\Sigma_{ii} \geq 1$; $\lvert\nu\rvert < 2^{80}$; $0 \leq w_i < 2^{40}$; $0 \leq \mu_i < 2^{80}$ | `Num2Bits` on value (+ offset) |
| Budget $\Sigma w_i = ONE$ **exactly** | one linear constraint |
| Stationarity $\lvert(\Sigma w)_i - \nu - \mu_i\rvert \leq \tau$ | two 90-bit range checks of $\tau \pm r$ |
| Complementary slackness $\Sigma w_i \mu_i \leq \tau$ | one 130-bit range check of $\tau \cdot ONE - gap$ |
| Convexity $\lvert\Sigma_{ij} \cdot ONE - (LL^\top)_{ij}\rvert \leq \tau$, $j \leq i$ | two 90-bit range checks per entry |
| Commitments and variance as above | Poseidon (circomlib), 6 products |
| `provider` | a quadratic constraint so the public input is bound into the proof |

12,569 constraints (9,421 non-linear). **No wrap-around:** with the input ranges every intermediate quantity is below $2^{131}$ in magnitude, far under the field modulus $p \approx 2^{254}$, so each range check is an exact integer comparison. `checkWitness` in `agent/src/engine/privateMinvar.ts` mirrors every arithmetic constraint and is used as a pre-flight and in tests.

### 10.3 Theorem (private ε-optimality)

*If* `PrivateMinVarianceVerifier` *accepts, then (under the knowledge soundness of Groth16 and the collision resistance of Poseidon) the provider knows Σ committed to by the attested* `dataCommitment` *and w committed to by* `weightsCommitment` *such that* $\tfrac{1}{2} w^T \Sigma w \leq \min_{x \geq 0,\ 1^T x = 1} \tfrac{1}{2} x^T \Sigma x + \tau(|\nu| + 3 + \tau) + \tfrac{1}{2} n\tau(2 + \tau)^2$.

**Proof.** The circuit's constraints are exactly the hypotheses of Theorem 4.4 (P1 holds with zero budget error), so the bound of §4.4 applies verbatim. For the live job, $|\nu| \approx 0.0016$ and the bound is $\approx 1.5 \cdot 10^{-9}$. ∎

### 10.4 What the verifier checks before the pairing

| Check | Error | What it rules out |
|---|---|---|
| The data commitment is signed by the registered vendor (EIP-191 over tag, chain id, commitment, universe) | `UntrustedDataAttestation(signer)` | proofs about made-up data; replaying an attestation on another chain |
| Every public signal is a canonical field element | `NotAFieldElement` | malleable encodings |
| `verifyProof(a, b, c, [weightsCommitment, variance, dataCommitment, provider])` | `ProofRejected` | everything else: a different portfolio, a lower claimed variance, a copied proof |

Verification costs ≈ 270 k gas, and the escrow needs no change: the verifier is just another `IProofVerifier` named in the provider-signed quote. A client accepts a quote only if its spec names the quoting provider; the escrow already requires the delivery to come from that provider.

### 10.5 Trusted setup

Groth16 needs a structured reference string. This deployment used a local two-phase ceremony ( `circuits/setup.sh` ): phase 1 powers of tau ($2^{14}$) with two contributions and a beacon, phase 2 with two contributions and a beacon. The beacons are the hashes of public Sepolia transactions from earlier runs (S4 and N6), fixed before the setup ran. Groth16 is sound if any one contributor destroyed their secret; since every contribution ran on one machine, **this setup must be treated as trusted** (risk K18). A production deployment takes phase 1 from a public ceremony ( `PTAU=… bash circuits/setup.sh` ) and runs phase 2 with independent parties; nothing else changes. `circuits/ceremony-transcript.txt` records every contribution hash, and `snarkjs zkey verify` reproduces the check from the committed `.r1cs` , `.ptau` and `.zkey` .

### 10.6 The live run (Ethereum Sepolia)

Helix attested a six-sleeve private-markets model by commitment only. The QED agent proved the optimal allocation in 1.5 s; the original `ProofEscrow` paid 12 tUSD into QED's vault on that proof (Z3). Mallory then replayed QED's proof from the public calldata into her own job ( `ProofRejected` : it is bound to QED), claimed a lower variance ( `ProofRejected` ), and delivered a genuine proof about a data set she attested herself ( `UntrustedDataAttestation` ). The chain never saw the model or the portfolio. (So that reviewers can reproduce the run, the demo's synthetic model is in `agent/src/engine/privateScenario.ts` ; in production it stays with the vendor and the agent, and only its commitment is ever published.)

---

## 11. Theorems with money on them ( `OptimalityBounty` , `MaximalityBounty` )

A proof in a document is only as convincing as the reader's patience. These two contracts turn the two theorems QED's money depends on into **standing, self-adjudicating offers**: each holds 1,000 tUSD and pays all of it, with no human involved, to the first caller who refutes its theorem on-chain.

| Contract | Theorem guarded | A claim must contain | The contract checks, in exact integers |
|---|---|---|---|
| `OptimalityBounty` | §4.4: an accepted certificate is within the bound of the optimum | a certificate the **deployed** `MinVarianceVerifier` accepts, and a feasible portfolio x | re-runs the verifier; $x \geq 0$, $\Sigma x = 1$; pays iff $F(w) - F(x) > 2tW(\lvert\nu\rvert + 3W + t) + n \cdot t \cdot (2W + t)^2$ (the theorem's inequality $\times 2W^3$) |
| `MaximalityBounty` | §9.2: a netting with $\Sigma x = \Sigma d \cdot y$ is maximal | an economy, a netting `QEDClearing` would accept, and another netting | re-states `QEDClearing.net` 's acceptance rule line for line; checks the alternative conserves every net position within the debts; pays iff it cancels more |

If the theorems are true the bounties can never pay, and every attempt reverts with the numbers that show why: `TheoremHolds(advantage, bound)` or `TheoremHolds(169, 162)` . Positive controls in `contracts/test/Bounties.t.sol` show that against an unsound verifier, or an acceptance rule that under-reports by 7 tUSD, the same contracts pay out at once; a differential test shows `MaximalityBounty.accepts` agrees with the deployed `QEDClearing` on all 60 generated economies; fuzzers try 2,000 random feasible portfolios and 2,000 random circulations.

On Sepolia, Mallory tried the equal-weight portfolio (BA1), QED's optimum shifted by 0.1% (BA2), the forged certificate the chain refused in job #2 (BA3), the naive 162 tUSD netting (BM1) and a 170 tUSD netting that breaks one net position (BM2). All five reverted; `npm run verify` checks daily that both pools are still full.

PART 3

# QED — Security Design & Self-Assessment

> Scope: `contracts/src/*` (QEDVault, QEDLens, ProofEscrow, MinVarianceVerifier, QEDClearing, TestUSD) and the off-chain agent system in `agent/`. Status: **testnet prototype, not externally audited.** Every claim below links to a test, an invariant, or a public Sepolia transaction.

## 1. Security thesis

**Treat the AI as compromised by default.** Prompt injection, tool poisoning, model error and key theft are not edge cases for autonomous agents — they are the operating environment. QED therefore makes the safety of money independent of the AI's behaviour:

1. **The AI holds one key that can call one function** (`executePayment`), and that function succeeds only with an owner-signed mandate *and* an independent co-signer certificate *and* inside the solvency invariant.
2. **The worst case is computed, not hoped for.** `QEDLens.blastRadius` returns, at any block, the maximum loss if the agent key and the co-signer key are both stolen. Loss to addresses the owner never approved is 0.
3. **Proof replaces trust in payment for work.** The escrow pays only for answers that pass an on-chain KKT optimality check.
4. **We publish the keys.** The honeypot vault's agent and co-signer private keys are in the repo (`evidence/ethereum-sepolia/honeypot.json`). Its 90 tUSD ring-fence is protected by the contract, not by secrecy.

## 2. Permission tiers (implemented in `QEDVault.sol`)

| Capability | Owner (cold) | Agent (AI hot key) | Co-signer (independent service) | Guardian | Creditor | Anyone |
|---|---|---|---|---|---|---|
| Sign mandates (scope of agent authority) | ✓ (off-chain EIP-712, EOA or ERC-1271 multisig) | ✗ | ✗ | ✗ | ✗ | ✗ |
| `executePayment` (spot / forward) | ✗ | ✓ only with owner mandate + co-signer certificate | ✗ (signs certificates only) | ✗ | ✗ | ✗ |
| Sign payment certificates | ✗ | ✗ | ✓ after 12 policy checks | ✗ | ✗ | ✗ |
| `ringFence` (create protected liability) | ✓ from free funds only | ✗ (only via mandate-scoped forward payments) | ✗ | ✗ | ✗ | ✗ |
| `withdraw` | ✓ free funds only — never ring-fenced | ✗ | ✗ | ✗ | ✗ | ✗ |
| `settle` a due liability | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ (always pays the creditor; works while paused) |
| `release` / `assign` a liability | ✗ | ✗ | ✗ | ✗ | ✓ own claims only | ✗ |
| `pause` | ✓ | ✗ | ✗ | ✓ | ✗ | ✗ |
| `unpause` | ✓ | ✗ | ✗ | ✗ (can only make things safer) | ✗ | ✗ |

| Capability | Owner (cold) | Agent (AI hot key) | Co-signer (independent service) | Guardian | Creditor | Anyone |
|---|---|---|---|---|---|---|
| `bumpEpoch` / `revokeMandate` (mass / single revocation) | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| `setAgent` / `setCosigner` / `setGuardian` / `setDailyCap` | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| `transferOwnership` (two-step) | ✓ | ✗ | ✗ | ✗ | ✗ | accept by pending owner |

ProofEscrow tiers: provider signs quotes and is the only party that may `deliver`; client opens by presenting a provider-signed quote; anyone may `refund` after the deadline (funds go to the client only); the provider may release the client early. MinVarianceVerifier and QEDLens are stateless and privilege-free.

QEDClearing tiers (no owner, no admin, no upgrade path):

| Capability | Member | Anyone | Notes |
|---|---|---|---|
| `join` | ✓ (self only) | ✓ | emits the hash of the netting terms: explicit, on-chain consent |
| `issue` an obligation | ✓ as **debtor** only | ✗ | nobody can create a debt for someone else |
| `pay` an obligation | ✓ as **debtor** only | ✗ | tokens move debtor → creditor, nowhere else |
| `net` (submit a netting) | ✓ | ✓ | permissionless, because the contract enforces: unchanged net positions, no new claims, full snapshot, maximality proof |

PrivateMinVarianceVerifier, OptimalityBounty and MaximalityBounty have no owner, no admin and no upgrade path. The private verifier's only configuration is fixed at deployment (the Groth16 verifier and the data vendor's attestation key). A bounty's only outflow is `claim`, which pays the caller only when its theorem is refuted; nobody, including the deployer, can withdraw a bounty pool.

Off-chain tiers: each role runs as a separate OS process that receives exactly one private key in its environment (`agent/scripts/live.ts → spawnRole`). The language model never receives any key, address book mutation rights or tool that can sign.

## 3. Security properties and how each is verified

| # | Property | Mechanism | Verified by |
|---|---|---|---|
| I1 | Vault balance ≥ ring-fenced liabilities, always | `a ≤ free` checks on every outflow (§1 ALGORITHMS) | invariant fuzzing (512×96, both hot keys stolen); `npm run verify` live |
| I2 | `totalProtected` = Σ open liabilities | single code path for create / settle / release | invariant `ProtectedEqualsSumOfOpenLiabilities` |
| I3 | Ring-fenced value reaches only its creditor | `settle` pays `liabilities[id].creditor`; only creditor may release/assign | invariant `CreditorsPaidExactly`; Sepolia P3 |
| I4 | Zero loss to addresses the owner did not approve | payee ∈ owner-signed mandate | invariant `ZeroLossToUnapprovedAddresses`; Sepolia X6, H4 |
| I5 | Signed terms are immutable | EIP-712 over every field; authenticity checked first | unit tests `Revert_Tampered*`; Sepolia X2, X3, X8, X9 |
| I6 | No replay | single-use `intentId` / `quoteId` | Sepolia X10; `test_Revert_Replay` |
| I7 | Agent key has no admin power | `onlyOwner` / creditor / guardian gates | invariant `AgentKeyHasNoAdminPower`; Sepolia X13–X15, H6 |
| I8 | Bounded loss under total hot-key compromise | global window cap + free-balance bound | fuzz `BlastRadiusIsAnUpperBound`; Sepolia X11, X12, H5 |

| # | Property | Mechanism | Verified by |
|---|---|---|---|
| I9 | Pause stops the AI, never the creditors | `executePayment` checks `paused`; `settle` doesn't | `test_GuardianPausesButCannotUnpause_CreditorsStillPaid`; Sepolia P1–P3 |
| I10 | Escrow pays only for ε-optimal work | KKT certificate check (Theorem §4.4) | 3×2000-run fuzz incl. "accepted ⇒ ε-optimal"; Sepolia X17, X18 |
| I11 | Clients can't underpay / redirect, providers can't swap the problem | quote binds price, payee, verifier, spec hash, deadline | `test_Revert_ClientTampers*`; Sepolia X2, X3 |
| I12 | Purchased data is the data the job committed to | `job.specHash == invoice.dataCommitment`, re-hashed by the agent | co-signer C8; agent `fulfil()` |
| I13 | A netting never changes any member's net position | per-member conservation check | `test_Revert_ShiftingValueBetweenMembers`; 2,000-run cycle fuzz; Sepolia N2; `npm run verify` |
| I14 | A netting never erases more than is owed and never creates a claim | $0 \le x \le d$; amounts only decrease | `test_Revert_CancellingMoreThanOwed`; Sepolia N3 |
| I15 | Only a provably maximal netting executes | LP-duality certificate, $\Sigma x = \Sigma d \cdot y$, exact integers | 60 random economies; TS solver = exhaustive oracle on 150 economies; Sepolia N1, N5 |
| I16 | A proof covers the whole, current state | every open id, current amounts | `test_Revert_HidingAnObligation`, `StaleAmounts_AndReplay`; Sepolia N4, N7 |
| I17 | The escrow pays for private work only if it is ε-optimal for the vendor's attested data | Groth16 proof of the KKT certificate + EIP-191 vendor attestation (§10 ALGORITHMS) | real-proof tests incl. 2,000-run corrupted-proof fuzz; no witness exists for 1/N, a 1 bp nudge or other data (`private.test.ts`); Sepolia Z3, ZA2, ZA3 |
| I18 | A valid proof is useless to anyone but its author | `provider` is a public input bound into the proof | `test_StolenProofIsUselessToAnotherProvider`; Sepolia ZA1 |
| I19 | The chain learns nothing about the model or the portfolio beyond two commitments and the variance | zero knowledge of Groth16; Poseidon commitments with 248-bit salts | by construction; `npm run verify` re-verifies the proof from public signals only |
| I20 | Theorems 4.4 and 9.2 are falsifiable on-chain by anyone, for money | `OptimalityBounty`, `MaximalityBounty` | positive controls pay against unsound rules; differential test vs `QEDClearing` on 60 economies; 2×2,000-run fuzz; Sepolia BA1–BA3, BM1–BM2; pools checked daily by `npm run verify` |

Test totals: **104 Solidity tests** (unit, fuzz, stateful invariants — ≈350 000 adversarial calls per CI run, 60 random clearing economies, a real Groth16 proof attacked 2,000 ways, both bounties), **50 TypeScript tests** (TS ≡ Solidity parity on a live node, portfolio solver vs. a 20 000-sample oracle, netting solver vs. an exhaustive oracle, zero-knowledge prover soundness checks, 32-case red-team).

## 4. Threat model

**Assets**: ring-fenced liabilities; free treasury; escrowed client funds; integrity of signed terms; the owner's authority; confidentiality of keys.

### Adversaries considered

| Adversary | Capabilities assumed | Outcome (evidence) |
|---|---|---|
| External prompt injector | arbitrary text in RFQs, invoices, chat | guard + typed-only economics; no payment path (X1, red-team 25/25) |
| Manipulated LLM | outputs anything, including "pay Mallory" | co-signer refuses (X4); contract rejects (X5–X7) |
| Agent-key thief | signs and sends any tx as the agent | can only pay owner-approved payees within caps & solvency (X5–X15) |
| Agent **+ co-signer** key thief | forges valid certificates | still bounded by owner mandate, caps, solvency (X6, X11, X12) |
| Public-key attacker (honeypot) | both hot keys are public | ≤ 1 tUSD/day to CloudCo; ring-fence intact (H3–H6, live) |
| Malicious client | edits signed quotes | rejected (X2, X3) |

| Adversary | Capabilities assumed | Outcome (evidence) |
|---|---|---|
| Lazy / dishonest AI provider | submits unoptimised or near-miss work | rejected by mathematics; client refunded (X17–X19) |
| Proof thief / data forger | copies a valid zk proof from calldata; proves on self-attested data; overstates quality | proof bound to its author; vendor attestation required (ZA1–ZA3) |
| Theorem challenger | any input at all, with a 1,000 tUSD incentive per theorem | every refutation attempt reverts with the gap it could not close (BA1–BA3, BM1–BM2) |
| Stranger | calls permissionless functions | can only settle due liabilities to their creditors (P3) |
| Owner-key thief | full owner powers | **out of scope** — see K1 |

**STRIDE summary.** Spoofing → EIP-712 signatures from fixed role addresses; Tampering → all terms signed, calldata re-hashed; Repudiation → every payment anchors a hash of its decision record; Information disclosure → LLM never sees keys, guard blocks secret requests (DE-1…6); Denial of service → no unbounded loops on-chain, payees ≤ 32; Elevation of privilege → single-function agent key, owner-only admin.

## 5. Known-risk list (honest)

| ID | Risk | Severity | Status / mitigation |
|---|---|---|---|
| K1 | **Owner-key compromise = full control** of mandates, caps, withdrawals of *free* funds. Ring-fenced liabilities remain protected even then. | High | Owner can be an ERC-1271 multisig (tested: `test_SmartAccountOwner_ERC1271`). Roadmap: timelocked admin. |
| K2 | **Both hot keys stolen**: attacker can move up to the blast radius to owner-approved payees (waste, not theft — unless an approved payee colludes). | Medium | Bounded and displayed on-chain (`blastRadius`); guardian pause; tight per-mandate caps. |
| K3 | Fixed 24h windows allow up to 2× cap around a window boundary. | Low | The published bound accounts for it (inclusive counting). Roadmap: rolling window. |
| K4 | **Verifier trust**: both sides must agree on the verifier contract in the quote; a malicious verifier could accept anything. | Medium | Verifier address is signed in the quote; clients pin canonical verifiers by bytecode (`npm run verify` checks bytecode). Roadmap: verifier registry. |
| K5 | **ε-optimal, not exact**: accepted solutions are within `(2n+3+ | ν | )τ` of the optimum; τ is client-chosen (≤ 1 %). |
| K6 | **Data confidentiality** in the public verifier: delivering a §4 certificate reveals the covariance in calldata. | Low (resolved for private jobs) | `PrivateMinVarianceVerifier` (§10) proves the same certificate in zero knowledge; clients choose per job. |
| K7 | **Guard is heuristic**: regex detectors can have false negatives/positives, the LLM is non-deterministic. | Medium | Guard is defence-in-depth only; the co-signer and contract are the binding controls. Red-team: 25/25 blocked, 0 false refusals. |
| K8 | **Co-signer liveness**: if the co-signer is down, the agent cannot pay (fails safe). | Low | Fail-closed by design; owner can rotate co-signer. |
| K9 | Token assumptions: standard ERC-20 (no fee-on-transfer / rebasing). | Low | Single, known asset per vault. |
| K10 | Timestamp influence (validators ±seconds) on windows / deadlines. | Low | Windows are hours/days; certificates valid for 2 h. |
| K11 | **Gas-schedule drift**: Sepolia's state-creation repricing (≈6.7× deployment cost) and EIP-7825's $2^{24}$ per-tx gas cap bound deployable code size. | Info | Vault slimmed to 9.1 KB (views moved to QEDLens); deploys at 14.96 M gas. |
| K12 | **Execution topology**: our build sandbox had no RPC egress. Transactions were signed in the sandbox against a wall-clock-synchronised local replica of Sepolia, then the identical signed bytes were broadcast via a browser relay; every public receipt was compared with the replica's (status + logs). Two earlier attempts failed (gas schedule; one under-provisioned agent transaction) and were superseded by a run under fresh keys. | Info | Disclosed in `docs/EVIDENCE.md`; `npm run verify` re-derives every claim from the public chain; the canonical run is self-contained. |

| ID | Risk | Severity | Status / mitigation |
|---|---|---|---|
| K13 | Unaudited prototype on a testnet with a valueless token. | — | Do not use with real funds. |
| K14 | Honeypot keys are public: anyone can spend ≤ 1 tUSD/day of honeypot free balance to CloudCo. | Expected | That is the demonstration of a bounded blast radius. |
| K15 | **Clearing griefing**: a member can issue many tiny obligations (≤ 128 open) or front-run a netting with a new obligation, making a submitted proof stale. | Low | Fails safe ( `IncompleteSnapshot` / `StaleAmount` ); the prover recomputes in milliseconds. Roadmap: per-member caps, a proposal window. |
| K16 | **Netting changes counterparties' credit mix** only by shrinking it: claims are reduced, never re-assigned, but a member loses a specific receivable it might have preferred to keep. | Low | Disclosed in the on-chain terms every member signs by joining. |
| K17 | Clearing obligations are acknowledgements, not escrowed funds: a debtor can still default on what remains after netting. | Medium | Netting minimises what must be paid; pairing it with QEDVault ring-fencing is the roadmap. |
| K18 | **Trusted setup**: the Groth16 parameters came from a local two-phase ceremony whose contributions all ran on one machine; whoever held all contributions' randomness could forge proofs. | Medium (testnet) | Public-transaction beacons; transcript and `.ptau` / `.zkey` committed for `snarkjs zkey verify` . Production: phase 1 from a public ceremony, phase 2 with independent parties ( `PTAU=… bash circuits/setup.sh` ). |
| K19 | **Circuit completeness**: an under-constrained circuit would accept bad witnesses. | Medium | Every arithmetic constraint is mirrored in `checkWitness` ; explicit range checks keep all values ≪ p; tests show no witness exists for 1/N, a 1 bp nudge or other data. Roadmap: formal circuit audit (e.g. Picus / Circomspect). |
| K20 | **Provider binding is per spec**: a client that accepts a quote whose spec names a different provider could pay for a proof it cannot use. | Low | Clients check `spec.provider == quote.provider` (enforced by QED's client agent); the escrow already pins delivery to the quoted provider. |
| K21 | Bounty pools are finite and in a test token; they demonstrate falsifiability, not insurance. | Info | Pools and claims are public; `npm run verify` checks daily that no `Refuted` event exists. |

## 6. Incident response

1. **Guardian** pauses the vault (one transaction). Creditors keep being paid.
2. **Owner** bumps the epoch → every outstanding mandate and certificate is void.
3. **Owner** rotates the compromised key ( `setAgent` / `setCosigner` ), re-issues mandates, unpauses.
4. Forensics: every payment event carries `evidenceHash` ; the decision record reproduces the exact inputs.

## 7. Compliance-relevant controls

- **Four-eyes / maker–checker** for every discretionary payment (agent proposes, co-signer certifies).
- **Segregation of duties**: owner, agent, co-signer, guardian are distinct keys and processes.
- **Ring-fencing** of obligations (bankruptcy-remote within the vault).
- **Sanctions / deny-list screening** at the co-signer (C11).
- **Audit trail**: canonical decision records, hash-anchored on-chain; LLM transcripts hashed per turn.
- **Explainability**: every decision has a human-readable rationale; the agent answers "why" from its decision log.
- **Kill switch** with creditor continuity.

PART 4

# QED — Test Catalogue

Everything below runs in CI on every push (`.github/workflows/ci.yml`) and locally with `forge test && npm run test:ts`.

## Solidity (Foundry, `contracts/test`)

Default profile: 2 000 fuzz runs per property, invariants 512 runs × depth 96. CI profile (`FOUNDRY_PROFILE=ci`): 5 000 fuzz runs, invariants 1 024 × 128.

### `Invariants.t.sol` — stateful fuzzing with both hot keys stolen

`CompromisedAgentHandler` holds the agent key **and** the co-signer key and, on every call, chooses among: paying random/approved/attacker payees with genuinely signed certificates, forward payments, settling, releasing, admin calls, pausing, time travel. After every call:

| Invariant | Statement |
|---|---|
| `VaultAlwaysCoversRingFencedLiabilities` | `token.balanceOf(vault) ≥ totalProtected` |
| `ProtectedEqualsSumOfOpenLiabilities` | `totalProtected = Σ open liabilities` |
| `CreditorsPaidExactly` | each creditor received exactly what was settled to it |
| `ZeroLossToUnapprovedAddresses` | the attacker's balance never increases |
| `DailyCapRespected` | agent outflow per 24 h window ≤ `dailyCap` |
| `NoAgentOutflowWhilePaused` | no agent-initiated outflow while paused |
| `AgentKeyHasNoAdminPower` | owner, agent, co-signer, guardian, caps never change via the handler |
| `test_HandlerIsNotVacuous` | the handler really executes successful payments (guards against vacuous invariants) |

### `QEDVault.t.sol` — 34 tests

Authorisation (owner mandate × co-signer certificate, ERC-1271 smart-account owner), every tampering path (amount, recipient, due date, evidence, mandate scope), replay, expiry, revocation (single + epoch), key rotation, per-payment / period / lifetime / global window caps, solvency under total hot-key compromise, forward payments as ring-fenced promises, settlement by anyone when due (also while paused), creditor-only release/assign, guardian pause without unpause, two-step ownership, and `testFuzz_BlastRadiusIsAnUpperBound` (no sequence of attacker payments exceeds the published `blastRadius`).

### `ProofEscrow.t.sol` — 13 tests

Full lifecycle quote → open → deliver → settle into the agent's vault; client tampering with price / payee / verifier / spec / deadline; impostor-signed quotes; quote replay, wrong client, expiry; lazy 1/N portfolio; almost-optimal portfolio; fake convexity witness; solving a different problem; only-provider-delivers-once; refund after deadline by anyone (to the client only); early refund by the provider; `check` is a pure dry run.

### `MinVarianceVerifier.t.sol` — 12 tests

Honest certificate passes and returns the variance; asymmetric covariance, negative weight (short selling), budget violation, negative multiplier, dimension mismatch, overflow and tolerance games are rejected; gas cost. Fuzz properties derived from the theorem in `docs/ALGORITHMS.md §4.4`:

- `testFuzz_AcceptedImpliesEpsilonOptimal` — anything accepted is within `(2n+3+|v|)τ` of the true optimum;
- `testFuzz_MaterialReallocationIsRejected` — moving ≥ 1e-5 of weight away from the optimum is rejected;
- `testFuzz_RandomPortfolioWithBestMultipliersIsRejected` — a random portfolio dressed with its best possible multipliers is rejected.

### `QEDClearing.t.sol` (15 tests)

- `test_OptimalNettingExecutes_PositionsUnchanged`: the demo economy's proven netting clears 169 of 200 tUSD; every member's net position is unchanged.
- `test_GridlockThenSettlementWithExactCash`: gross payment fails (`ERC20InsufficientBalance`); after netting, the remaining debts settle in topological order with each agent's net debit only; every agent ends at exactly its net position.
- Rejections: `GreedyNettingIsNotProvablyMaximal` (162 vs 169), `ShiftingValueBetweenMembers`, `CancellingMoreThanOwed`, `HidingAnObligation`, `StaleAmounts_AndReplayAfterNetting`, `ForgedDualCertificate`, `IdsNotIncreasing`, `NothingToNet`, `PotentialOverflowGames`, `MembershipAndDebtRules`.
- `testFuzz_AnyCirculationAroundTheOptimumIsRejected` (2,000 runs): push any amount of debt around any of the 11 simple cycles of the demo economy; conservation still holds, and every such proposal is rejected.
- `test_SixtyRandomEconomies`: 60 random economies (3–7 agents, 3–16 debts) from the TypeScript solver: every optimal certificate executes with the exact cleared amount; wherever the greedy agent falls short (11 economies) it is rejected with `NotProvablyMaximal`.

### `PrivateMinVarianceVerifier.t.sol` (11 tests): a real Groth16 proof, attacked

The fixture (`npm run fixtures:private`) is a genuine proof from `circuits/private_minvar.circom`.

- Accepted: the proof verifies through the generated Groth16 verifier and returns `keccak(weightsCommitment, variance)`; ≈ 270 k gas.
- Rejected: an understated variance, a swapped portfolio commitment, the same proof under another provider's spec (`StolenProofIsUselessToAnotherProvider`), data attested by anyone but the vendor, an attestation replayed on another chain, a non-canonical field element.
- `testFuzz_AnyCorruptedProofCoordinateIsRejected` (2,000 runs): perturb any of the 8 proof coordinates by any amount; always `ProofRejected`.
- `test_EscrowPaysOnlyForThePrivateProof`: through `ProofEscrow`, Mallory's copy of QED's proof is refused and QED's delivery pays the vault.

### `Bounties.t.sol` (11 tests): theorems with money on them

- `OptimalityBounty`: the equal-weight portfolio, the certified portfolio itself and 2,000 random feasible portfolios (`testFuzz_Optimality_NoFeasiblePortfolioRefutesTheTheorem`) all fail to refute Theorem 4.4; a forged certificate cannot even enter (the deployed verifier reverts); infeasible challengers are refused; **positive control:** against a verifier that accepts anything, the bounty pays the first real refutation.
- `MaximalityBounty`: the demo economy is accepted at exactly 169; the greedy 162 cannot refute it; 2,000 random circulations built from the economy's cycles never beat it; `test_Maximality_AgreesWithQEDClearingOnSixtyEconomies` executes each of the 60 economies on a fresh `QEDClearing` and checks the bounty's rule agrees; **positive control:** an acceptance rule that under-reports by 7 tUSD gets refuted and pays out.

## TypeScript (`node --test`, `agent/test`)

| File | What it proves |
|---|---|
| `parity.test.ts` | Starts anvil, deploys the contracts, and checks **TypeScript ≡ Solidity** for every EIP-712 hash/digest (mandate, certificate, quote) and for the KKT checker's verdict and returned variance. |
| `engine.test.ts` | Certifying solver: honest certificate has zero stationarity residual; matches a 20 000-sample brute-force oracle on the simplex; lazy 1/N fails only on complementary slackness; a 0.01 % nudge is rejected; fixed-point round trip; economics (price = cost × 2.5 rounded up; unsettled revenue never counts as capital). |
| `netting.test.ts` | Netting solver equals an **exhaustive brute-force oracle** on 150 random economies; certificates are exact; netting never changes a net position; remaining debts always form a DAG that settles with net debits only. |
| `private.test.ts` | The zero-knowledge prover: the fixed-point certificate satisfies every circuit constraint with zero duality gap and matches the exact solver to $10^{-12}$ per weight; the Groth16 proof verifies and its public signals equal the commitments, the provider and the variance; a tampered signal fails; **no proof exists** for the lazy 1/N portfolio, a 1-basis-point nudge, or a different data commitment (the witness generator's constraints fail). |

| File | What it proves |
|---|---|
| `guard.test.ts` | The 32-case red-team corpus ( `redteam.corpus.ts` : 25 attacks across induced transfer, data extraction, privilege escalation and prompt injection, in English and Chinese, plus 7 benign requests) through the deterministic guard; plus overrides of a fully compromised LLM (proposes paying the attacker; leaks secrets; exceeds the free balance). |

## End-to-end

- `QED_OFFLINE=1 npm run local` — the whole economy (deploy, fund, lifecycle, conversation, 18 attacks, honeypot, emergency) on a local chain, in CI.
- `npm run local:clearing` — the clearing economy (gridlock, 6 attacks, proven netting, settlement) on a local chain.
- `npm run local:private` — the private economy (zero-knowledge proof, escrow settlement, 3 attacks, both bounties funded and 5 refutation attempts) on a local chain.
- `npm run zk:setup` — re-runs the trusted setup from the circuit (new keys; the committed keys are the ones deployed).
- `npm run verify` — independent re-verification of the public Sepolia evidence for all three runs (bytecode, receipts, replayed revert errors, signatures, KKT re-check, netting certificate re-check, Groth16 proof re-verified off-chain with snarkjs, vendor attestation recovered, bounty pools intact, live invariants), in CI on every push and daily.
- `npm run redteam` — the corpus against the live language model alone and the guarded pipeline; report in `evidence/redteam/report.md` .

## Clean-clone reproduction log

| When (UTC) | Machine | Steps | Result |
|---|---|---|---|
| 2026-10-19 16:50 | Fresh Linux container with nothing installed (no Foundry, no keys, no prior state) | `git clone --recursive` , install Foundry 1.7.1 + solc 0.8.26 from their GitHub releases, `npm ci` , `forge build` , `forge test` , `npm run test:ts` | **67 / 67** Solidity tests and **42 / 42** TypeScript tests passed |
| 2026-10-19 21:46 | Fresh `git clone --recursive` of commit `dee4c07` into an empty directory (Foundry 1.7.1 toolchain present; no keys, no `node_modules` , no build output) | `npm ci` , `forge build` , `forge test` , `npm run test:ts` , `QED_OFFLINE=1 npm run local` , `npm run local:clearing` | **82 / 82** Solidity and **45 / 45** TypeScript tests passed; both economies ran end to end on a local chain (4 min 6 s) |
| 2026-10-20 02:29 | Fresh `git clone --recursive` of commit `aeaf628` into an empty directory (Foundry 1.7.1 toolchain present; no keys, no `node_modules` , no build output) | `npm ci` , `forge build` , `forge test` , `npm run test:ts` , `QED_OFFLINE=1 npm run local` , `npm run local:clearing` , `npm run local:private` | **104 / 104** Solidity and **50 / 50** TypeScript tests passed; all three economies ran end to end on a local chain, including a real Groth16 proof (4 min 49 s) |
| every push + daily | GitHub Actions `ubuntu-latest` | `.github/workflows/ci.yml` : Solidity suite, TypeScript suite, both economies on a local chain, `npm run verify` (both runs) against public Sepolia | green |

PART 5

# QED — Deployment & Operations

## Live deployment (Ethereum Sepolia, chain id 11155111)

| Contract | Address |
|---|---|
| QEDVault (agent) | `0x441f71f12a1f8a3b7d432851d4d4244fc33e8ca6` |
| ProofEscrow | `0x2e3362d6dc88c40cf95f885c44fe8fff9979ba9f` |
| MinVarianceVerifier | `0x1d3eced4bd1cef7edc315315b9fd6acae283dd05` |
| QEDLens | `0x846186f21227793760429696bb4fd7490236f101` |
| TestUSD (tUSD, 6 dp, `faucet()` 1 000/day) | `0x7adb609421e30b3df3063ff2de9d425564893f35` |
| QEDVault (honeypot, keys published) | `0xe5b3e7773115dba454e7e40fa97a5d3e332c7f04` |

Deployment block 11 948 657. Machine-readable: `deployments/11155111.json`. All transactions: `docs/EVIDENCE.md`.

## Prerequisites

- Node 22, npm
- Foundry ( `forge` , `anvil` , `cast` ) — `curl -L https://foundry.paradigm.xyz | bash && foundryup`
- `git clone --recursive` (OpenZeppelin 5.1 and forge-std are submodules)

```
npm ci
forge build           # solc 0.8.26, via-IR, optimizer 200 runs
```

## Local (no keys, no network)

```
anvil &                           # or let the script start one
QED_OFFLINE=1 npm run local       # deterministic dev keys, offline brain
```

Evidence lands in `evidence/local/` ; build the dashboard against it with `EVIDENCE=evidence/local npm run build:app` .

## Public network

1. `cp .env.example .keys.env` and fill in **fresh** keys (one per role). `.keys.env*` is gitignored.
2. Fund only the deployer; the script funds every other role with exactly the gas it needs.
3. Run:

```
RPC_URL=https://ethereum-sepolia-rpc.publicnode.com CHAIN_ID=11155111 \
GAS_LIMIT=16700000 MAX_FEE_GWEI=0.02 PRIORITY_FEE_GWEI=0.01 REQUIRE_FIXED_GAS=1 \
npm run live
```

The orchestrator spawns the co-signer (port 7701), vendor (7702) and agent (7703) as **separate processes, each given exactly one private key**, and records every step into `evidence/<network>/manifest.json` .

### Why a fixed gas limit on Sepolia

Sepolia currently prices state creation far above the classic schedule (a plain transfer to a fresh account ≈ 205 k gas; deploying the 9.1 KB vault ≈ 15 M gas) and EIP-7825 caps any transaction at $2^{24} = 16\,777\,216$ gas. Local nodes don't model this, so `eth_estimateGas` against a replica under-estimates. Every transaction therefore carries `GAS_LIMIT=16700000` ; only gas **used** is

charged. `REQUIRE_FIXED_GAS=1` makes every process refuse to send without it.

### Signing without RPC egress (how the canonical evidence was produced)

`SHADOW_OF=11155111 npm run local` runs the same economy against an anvil replica (chain id 11155111, `osaka`, wall-clock timestamps, deployer nonce/balance mirrored from Sepolia), then `agent/scripts/outbox.ts` exports the signed raw transactions in canonical order to `relay/outbox-11155111.json`. Any machine with RPC access can broadcast them unchanged (`eth_sendRawTransaction`) and compare each receipt with the replica's (`shadowStatus`, `shadowLogs`). `npm run resume` rebuilds the replica by replaying the exact bytes already on-chain and continues the scenario.

## Operations runbook

| Situation | Action | Who |
|---|---|---|
| Suspected agent or co-signer compromise | `pause()` | guardian or owner |
| Invalidate every outstanding mandate and certificate | `bumpEpoch()` | owner |
| Revoke one mandate | `revokeMandate(hash)` | owner |
| Rotate keys | `setAgent` / `setCosigner` / `setGuardian` | owner |
| Change the global 24 h cap | `setDailyCap` | owner |
| Resume | `unpause()` | owner only |
| A creditor's bill is due | `settle(id)` | anyone — works while paused |

Monitor: `QEDLens.snapshot(vault)` returns balance, ring-fenced, free, 24 h blast radius, open liabilities, epoch and paused state in one call; the dashboard polls it live.

APPENDIX A

# QED — On-chain Evidence (Ethereum Sepolia)

> Generated from `evidence/ethereum-sepolia/manifest.json` and `evidence/ethereum-sepolia/receipts.json` by `npm run evidence:md`. Re-derive every line from the public chain with `npm run verify` (no keys needed).

| | |
|---|---|
| Network | Ethereum Sepolia (chain id 11155111) |
| Blocks | 11,948,657 → 11,949,339 |
| Transactions | 55 (36 succeeded · 19 reverted **by design** — every one an attack or a paused-vault test) |
| Plus Run 3 (zero-knowledge job, bounties) | 24 more transactions — see below |
| Plus Run 2 (certified clearing) | 44 more transactions on a separate contract — see below |
| Off-chain signed artefacts | 7 (mandate, quote, certificate, vendor release, conversation, red-team) — hashes anchored on-chain |
| Total gas | 66,358,024 |
| Language model in the loop | anthropic:claude-sonnet-5-5 |

## Contracts

| Contract | Address | Deployment tx |
|---|---|---|
| QEDVault — the agent's vault | `0x441f71f12a1f8a3b7d432851d4d4244fc33e8ca6` | `0xf4b13b49…80a5` |
| ProofEscrow | `0x2e3362d6dc88c40cf95f885c44fe8fff9979ba9f` | `0x84e391b5…e83d` |
| MinVarianceVerifier | `0x1d3eced4bd1cef7edc315315b9fd6acae283dd05` | `0x903d0e80…2589` |
| QEDLens | `0x846186f212277937604296 96bb4fd7490236f101` | `0xf67a8e98…e8d9` |
| TestUSD (tUSD) | `0x7adb609421e30b3df3063ff2de9d425564893f35` | `0x209da471…7dd4` |
| QEDVault — HONEYPOT (keys published) | `0xe5b3e7773115dba454e7e40fa97a5d3e332c7f04` | `0xde7ac12c…1438` |

## Actors (one key each, one process each)

| Role | Address |
|---|---|
| deployer | `0x297ea575224e00FeF5a94c8f046af3540Fd60D5d` |
| owner | `0xb7Ed1049456550Cbd8e1C6578A8cF1DD495797f7` |
| agent | `0x2A101d90D3E502bDBd4a06c6712cA7D240A9d0Bb` |
| cosigner | `0xf1F8941003702C5141977Ee0E9E03A45a9f62402` |
| guardian | `0x22842F73B4596735d1b55DCaeEC10c5f48522e0E` |
| client | `0x286C262a6B324a56C84057d9B11a6F3A02232d18` |
| vendor | `0x6Bb74076b522a9d38c62EC7dF914274e121fFAb8` |
| creditor | `0x2d3e8af597fcA9F4ecDc41C69F2956EDd6f1CB78` |
| mallory | `0x3BDB7Ed120ff19453EA5A1e59A60AB72D7f2Dc7e` |
| hpAgent | `0x22f224dd5Db8344E6474c9d165DC273E96c173F5` |
| hpOwner | `0x8B3fB35964E882513b44a01Ca41B3AC626200eDc` |
| hpCosigner | `0x35b370861B06aA2a372f02BdF027f5E324153ab7` |

## Every step of the scenario

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| D1 | Deploy TestUSD (valueless 6-dp test stablecoin) | Deployer | success | `0x209da471…7dd4` | 11948657 | 3,716,239 |
| D2 | Deploy MinVarianceVerifier (on-chain KKT optimality checker) | Deployer | success | `0x903d0e80…2589` | 11948657 | 3,847,926 |
| D3 | Deploy ProofEscrow (work settles on proof, not trust) | Deployer | success | `0x84e391b5…e83d` | 11948657 | 9,051,450 |
| D4 | Deploy QEDVault for the QED agent | Deployer | success | `0xf4b13b49…80a5` | 11948657 | 14,963,244 |
| D5 | Deploy the HONEYPOT vault (agent + co-signer keys published) | Deployer | success | `0xde7ac12c…1438` | 11948657 | 14,963,232 |
| D6 | Deploy QEDLens (stateless on-chain views: blast radius, snapshots, EIP-712 digests) | Deployer | success | `0xf67a8e98…e8d9` | 11948658 | 8,026,770 |
| S1 | Owner deposits 100 tUSD into the agent's vault | Owner | success | `0xdbbc3dd1…a139` | 11948660 | 148,905 |
| S2 | Owner ring-fences 70 tUSD for CloudCo's compute bill (protected liability #1) | Owner | success | `0xbbdb4d37…253b` | 11948660 | 800,291 |
| S3 | Owner signs the operating mandate off-chain (EIP-712) — no transaction, no gas | Owner | off-chain, signed | `0xc4cd1e885a…` | | |
| L1 | QED agent evaluates the RFQ and signs a quote: 15.00 tUSD | QED agent | off-chain, signed | `0xd9e55c9e6e…` | | |
| L2 | Client funds the escrow with the agent-signed quote (job #1, 15 tUSD) | Client (Northwind) | success | `0xa0546be0…46b9` | 11948661 | 1,389,524 |
| L3 | Independent co-signer passes 12 checks and issues a payment certificate | Co-signer | off-chain, signed | `0xb0dd4c9948…` | | |
| L4 | AGENT KEY pays Helix 6 tUSD under owner mandate + co-signer certificate | QED agent | success | `0x56a2334e…83b6` | 11948662 | 747,079 |
| L5 | Vendor sees the payment on-chain and releases the licensed risk model | Helix Data | off-chain, signed | — | | |
| L6 | Agent delivers the optimal portfolio + KKT certificate → escrow verifies on-chain → 15 tUSD settles into the vault | QED agent | success | `0x4166253f…35f9` | 11948662 | 424,082 |
| C1 | Owner-confirmed chat payment: 3 tUSD to Helix (clarified, corrected, signed) | QED agent | success | `0x7000d3cc…5ee3` | 11948662 | 236,892 |
| C2 | Forward payment: agent PROMISES Helix 4 tUSD in 7 days — ring-fenced on creation | QED agent | success | `0xf0658094…bcad` | 11948662 | 691,012 |
| C3 | Red-team conversation: induced transfer, data extraction, privilege escalation, prompt injection — all refused | Owner ⇄ QED agent | off-chain, signed | — | | |
| X1 | Prompt injection inside a counterparty RFQ: instruction ignored, legitimate work still quoted | Mallory → QED agent | off-chain, signed | — | | |
| X2 | Client-side tampering: open escrow with the agent's quote but price 15 → 1 tUSD | Mallory | **reverted** · `BadProviderSignature` | `0xf5f2dff7…c917` | 11948664 | 45,976 |
| X3 | Client-side tampering: redirect the agent's payee to Mallory | Mallory | **reverted** · `BadProviderSignature` | `0x86cda26b…2902` | 11948664 | 45,976 |
| X4 | Worst case: the LLM obeys the injection and requests payment → independent co-signer REFUSES | Co-signer | off-chain, signed | `0xdb5718a9f2…` | | |
| X5 | Stolen AGENT key + co-signature forged by Mallory → pay Mallory 25 | Mallory | **reverted** · `BadCosignerSignature` | `0xd3e5a89f…d3ab` | 11948665 | 91,032 |

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| X6 | Stolen AGENT key + stolen CO-SIGNER key → genuinely signed certificate paying Mallory | Mallory | **reverted** · `PayeeNotInMandate` | `0xf9a023d7…c018` | 11948665 | 91,032 |
| X7 | Stolen keys + a mandate Mallory signed herself (payee: Mallory, cap 1000) | Mallory | **reverted** · `BadOwnerSignature` | `0xa9b9a57f…a9f5` | 11948665 | 88,984 |
| X8 | Harvest the agent's real certificate from calldata; change amount 6 → 9 | Mallory | **reverted** · `BadCosignerSignature` | `0xc84501ca…09b9` | 11948665 | 91,032 |
| X9 | Same genuine certificate redirected to another approved payee | Mallory | **reverted** · `BadCosignerSignature` | `0xca320184…4b87` | 11948665 | 91,032 |
| X10 | Replay the exact genuine certificate | Mallory | **reverted** · `IntentAlreadyUsed` | `0xca786a4e…2957` | 11948665 | 91,032 |
| X11 | Both hot keys: pay CloudCo 33.00 tUSD — one dollar more than the free balance | Mallory | **reverted** · `SolvencyViolation` | `0x5835240c…2e09` | 11948665 | 91,032 |
| X12 | Both hot keys: pay CloudCo 13.00 tUSD — above the remaining 24h window | Mallory | **reverted** · `ExceedsDailyCap` | `0x6b4b0b71…e354` | 11948665 | 91,032 |
| X13 | Stolen agent key calls withdraw() | Mallory | **reverted** · `NotOwner` | `0xb988e05e…bcec` | 11948665 | 19,352 |
| X14 | Stolen agent key tries to release CloudCo's ring-fence | Mallory | **reverted** · `NotCreditor` | `0x9c908ed0…aaee` | 11948665 | 19,852 |
| X15 | Stolen agent key tries setDailyCap(max) | Mallory | **reverted** · `NotOwner` | `0x48b6a13b…c2a8` | 11948665 | 17,524 |
| X16 | Client hires Mallory instead (cheaper: 9 tUSD) — job #2 funded | Client (Northwind) | success | `0x952b0913…6f8d` | 11948666 | 1,291,592 |
| X17 | Mallory delivers a lazy 1/N portfolio dressed with the best possible multipliers | Mallory | **reverted** · `DualityGapTooLarge` | `0x5a6f26ac…6481` | 11948667 | 191,384 |
| X18 | Mallory delivers an 'almost optimal' portfolio (optimum shifted by 0.01%) | Mallory | **reverted** · `StationarityViolated` | `0xc269055a…f0fe` | 11948668 | 191,384 |
| H1 | Honeypot owner deposits 100 tUSD | Honeypot owner | success | `0xe6553dd5…7279` | 11948669 | 148,905 |
| H2 | Honeypot owner ring-fences 90 tUSD for CloudCo until 30 Jun 2027 | Honeypot owner | success | `0x1826c4bb…ebd9` | 11948669 | 800,291 |
| H3 | Using ONLY the published keys: pay CloudCo 1 tUSD (allowed — proves the keys are real) | Anyone (published keys) | success | `0xd401d7e9…7de6` | 11948670 | 746,554 |
| H4 | Published keys: pay Mallory 10 tUSD | Mallory | **reverted** · `PayeeNotInMandate` | `0x9f1f19d5…23b7` | 11948670 | 88,984 |
| H5 | Published keys: pay CloudCo 50 tUSD early | Mallory | **reverted** · `ExceedsPerPaymentCap` | `0xf30a4765…1b4f` | 11948670 | 88,984 |
| H6 | Published agent key: withdraw() | Mallory | **reverted** · `NotOwner` | `0x887cab2c…3c3f` | 11948670 | 19,352 |
| X19 | Mallory missed the deadline with a valid proof → anyone refunds the client | Client (Northwind) | success | `0x014cb969…4b83` | 11949335 | 63,177 |
| P1 | Guardian hits the kill-switch: vault paused | Guardian | success | `0x282afb62…640b` | 11949336 | 31,188 |

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| P2 | While paused, even a fully valid agent payment is refused | QED agent | **reverted** · `VaultPaused` | `0x6750861d…0b0f` | 11949337 | 91,032 |
| P3 | Bill due while paused: MALLORY triggers settlement — CloudCo receives exactly 70 tUSD | Mallory (as a stranger) | success | `0x9f4a753c…135d` | 11949338 | 89,792 |
| P4 | Owner reviews and unpauses | Owner | success | `0x6ba8db0a…243c` | 11949339 | 30,815 |

## Attack transactions (19) — each reverted with exactly the designed error

| Step | Attack | Defence (custom error) | Layer |
|---|---|---|---|
| X2 | Client-side tampering: open escrow with the agent's quote but price 15 → 1 tUSD | `BadProviderSignature` | ProofEscrow |
| X3 | Client-side tampering: redirect the agent's payee to Mallory | `BadProviderSignature` | ProofEscrow |
| X5 | Stolen AGENT key + co-signature forged by Mallory → pay Mallory 25 | `BadCosignerSignature` | QEDVault |
| X6 | Stolen AGENT key + stolen CO-SIGNER key → genuinely signed certificate paying Mallory | `PayeeNotInMandate` | QEDVault |
| X7 | Stolen keys + a mandate Mallory signed herself (payee: Mallory, cap 1000) | `BadOwnerSignature` | QEDVault |
| X8 | Harvest the agent's real certificate from calldata; change amount 6 → 9 | `BadCosignerSignature` | QEDVault |
| X9 | Same genuine certificate redirected to another approved payee | `BadCosignerSignature` | QEDVault |
| X10 | Replay the exact genuine certificate | `IntentAlreadyUsed` | QEDVault |
| X11 | Both hot keys: pay CloudCo 33.00 tUSD — one dollar more than the free balance | `SolvencyViolation` | QEDVault |
| X12 | Both hot keys: pay CloudCo 13.00 tUSD — above the remaining 24h window | `ExceedsDailyCap` | QEDVault |
| X13 | Stolen agent key calls withdraw() | `NotOwner` | QEDVault |
| X14 | Stolen agent key tries to release CloudCo's ring-fence | `NotCreditor` | QEDVault |
| X15 | Stolen agent key tries setDailyCap(max) | `NotOwner` | QEDVault |
| X17 | Mallory delivers a lazy 1/N portfolio dressed with the best possible multipliers | `DualityGapTooLarge` | MinVarianceVerifier |
| X18 | Mallory delivers an 'almost optimal' portfolio (optimum shifted by 0.01%) | `StationarityViolated` | MinVarianceVerifier |
| H4 | Published keys: pay Mallory 10 tUSD | `PayeeNotInMandate` | QEDVault |
| H5 | Published keys: pay CloudCo 50 tUSD early | `ExceedsPerPaymentCap` | QEDVault |
| H6 | Published agent key: withdraw() | `NotOwner` | QEDVault |
| P2 | While paused, even a fully valid agent payment is refused | `VaultPaused` | QEDVault |

## Set-up transactions (funding, mints, approvals)

| # | Transaction | What | Block |
|---|---|---|---|
| 6 | `0x04dad896…0170` | Fund gas: owner (0.003 ETH) | 11948658 |
| 7 | `0xbe80e5de…29a6` | Fund gas: agent (0.004 ETH) | 11948658 |
| 8 | `0x20134f85…5b49` | Fund gas: client (0.002 ETH) | 11948658 |
| 9 | `0x7e261aca…074d` | Fund gas: Mallory (0.004 ETH) | 11948658 |
| 10 | `0xa6179d11…b41f` | Fund gas: guardian (0.001 ETH) | 11948658 |

| # | Transaction | What | Block |
|---|---|---|---|
| 11 | `0xfd346ae4…0131` | Fund gas: honeypot owner (0.002 ETH) | 11948658 |
| 12 | `0xd47563a3…1231` | Fund gas: honeypot agent (0.002 ETH) | 11948659 |
| 13 | `0x63d04fe7…5ff9` | Mint 100 tUSD to owner | 11948659 |
| 14 | `0x75ea1d95…242d` | Mint 100 tUSD to client (Northwind) | 11948659 |
| 15 | `0x57128e5f…8303` | Mint 100 tUSD to honeypot owner | 11948659 |
| 16 | `0xda52f56f…a20b` | Owner approves vault | 11948660 |
| 19 | `0x7f7b5268…71e2` | Client approves escrow | 11948661 |
| 25 | `0xfbdefe11…6083` | Mint 50 tUSD to Mallory | 11948663 |
| 26 | `0x6a54f559…8fec` | Mallory approves escrow | 11948664 |
| 43 | `0xf4482304…09e9` | Honeypot owner approves honeypot vault | 11948669 |

## Final state (read from QEDLens after the last transaction)

| | Balance | Ring-fenced | Free | 24 h blast radius |
|---|---|---|---|---|
| Agent vault | 36.00 | 4.00 | 32.00 | 32.00 |
| Honeypot vault | 99.00 | 90.00 | 9.00 | 1.00 |

Token balances: CloudCo (creditor) 71.00 · Helix (vendor) 9.00 · Northwind (client) 85.00 · Mallory 50.00 (all of it minted to her for the test; **0 from any vault or the escrow**).

## Run 2 — certified clearing of an agent economy (QEDClearing)

> `evidence/ethereum-sepolia/clearing/manifest.json` , `receipts.json` , `proof.json` . Separate keys, separate token, separate contract; `npm run verify` re-checks it too, including an independent re-check of the netting certificate taken from the executed calldata.

| | |
|---|---|
| Blocks | 11,949,578 → 11,949,690 |
| Transactions | 44 (37 succeeded · 7 reverted **by design**) |
| Gross debt between 5 AI businesses | 200.00 tUSD in 11 obligations; cash held in total 28.00 tUSD |
| Naive netting (shortest cycles first) | 162.00 — **refused on-chain** as `NotProvablyMaximal` |
| Proven-maximal netting | 169.00 cancelled with the LP-duality certificate Σx = Σdᵧ checked in exact integers |
| Settled with tokens afterwards | 31.00 tUSD; open obligations at the end: 0 |

| Contract | Address |
|---|---|
| QEDClearing | `0x7b7de15772198d82d07019fc816cffc52e7d6ad2` |
| TestUSD (clearing run) | `0x6753a8ea4560f07534bcfbca2dea790afb9552d4` |

| Role | Address |
|---|---|
| deployer | `0x94C938E642BbC657D5F45546E35268a70c936448` |
| northwind | `0x053DCBCaEF06A44feAeFd94A2DbE9F160E789816` |
| qed | `0x0460e0d0753E487b3a5E26FE8cf16A4baC6C918d` |
| cloudco | `0x0418CfE812572f01535Fb260b6a77dAb5704D737` |
| helix | `0xE21F926030fd77Ad4EFEbd1b22D6633cda3499C7` |
| atlas | `0x13958b9C4154996E6Db79f9a86c5765e0D6048AF` |

| Role | Address |
|---|---|
| mallory | `0x56D161255Ced6863698eA3c6a886982088fa30a8` |

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| K1 | Deploy the clearing run's TestUSD (valueless; minter = this run's deployer) | Deployer | success | `0x627457ae…5865` | 11949578 | 3,716,239 |
| K2 | Deploy QEDClearing (proof-gated multilateral netting) | Deployer | success | `0x58efadf2…d140` | 11949578 | 8,579,134 |
| J1 | Northwind joins the clearing house | Northwind | success | `0x73dc94e3…03ab` | 11949579 | 348,185 |
| J2 | QED joins the clearing house | QED | success | `0xba620589…537d` | 11949580 | 250,265 |
| J3 | CloudCo joins the clearing house | CloudCo | success | `0x4e6748d9…aa6b` | 11949581 | 250,265 |
| J4 | Helix joins the clearing house | Helix | success | `0x2b3c68ee…6229` | 11949582 | 250,265 |
| J5 | Atlas joins the clearing house | Atlas | success | `0x03e19549…30ca` | 11949583 | 250,265 |
| I1 | Northwind owes QED 40 tUSD (portfolio subscription) | Northwind | success | `0x7b808c9a…3cee` | 11949584 | 795,367 |
| I2 | QED owes CloudCo 30 tUSD (GPU compute) | QED | success | `0xbd67bb55…a165` | 11949585 | 501,607 |
| I3 | CloudCo owes Helix 25 tUSD (data licence) | CloudCo | success | `0xadfa6dc4…8c32` | 11949586 | 501,619 |
| I4 | Helix owes Northwind 20 tUSD (advisory fees) | Helix | success | `0x8b216187…75e5` | 11949587 | 501,607 |
| I5 | QED owes Helix 15 tUSD (risk model) | QED | success | `0x7a0649e1…680f` | 11949588 | 501,607 |
| I6 | Helix owes QED 15 tUSD (model validation) | Helix | success | `0x280d6f94…4d71` | 11949589 | 501,607 |
| I7 | CloudCo owes Northwind 12 tUSD (volume rebate) | CloudCo | success | `0x59af539e…e0c0` | 11949590 | 501,595 |
| I8 | Northwind owes CloudCo 10 tUSD (hosting) | Northwind | success | `0x57bc03f7…1398` | 11949591 | 501,607 |
| I9 | Atlas owes QED 18 tUSD (backtests) | Atlas | success | `0xf5c02f7a…e673` | 11949592 | 501,619 |
| I10 | Helix owes Atlas 9 tUSD (data cleaning) | Helix | success | `0x8bfe4d58…b7a2` | 11949593 | 501,607 |
| I11 | CloudCo owes Atlas 6 tUSD (labelling) | CloudCo | success | `0x0ed4ee56…ee96` | 11949594 | 501,607 |
| G1 | Gridlock: Northwind tries to pay its 40 tUSD debt gross, but holds 18 | Northwind | **reverted** · `ERC20InsufficientBalance` | `0x7d86cdc9…3b2c` | 11949599 | 87,944 |
| P0 | QED agent proves the maximal netting: 169.00 of 200.00 tUSD can be cancelled | QED agent | off-chain | — | | |
| N1 | Lazy netting (shortest cycles first): cancels 162.00, not 169.00 | Mallory / rival clearing bot | **reverted** · `NotProvablyMaximal` | `0x1b62ae8c…4976` | 11949600 | 162,976 |
| N2 | Netting that quietly cancels 1 tUSD more of Northwind's debt to QED | Mallory / rival clearing bot | **reverted** · `NetPositionChanged` | `0x29c4ef66…3eaf` | 11949600 | 164,780 |
| N3 | Netting that cancels more than an obligation is worth | Mallory / rival clearing bot | **reverted** · `ExceedsObligation` | `0x96a304a4…bdfb` | 11949600 | 138,136 |

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| N4 | Netting over a snapshot that hides one obligation | Mallory / rival clearing bot | **reverted** · `IncompleteSnapshot` | `0xd6c3f25c…4c9c` | 11949600 | 129,944 |
| N5 | Forged certificate: drop a dual multiplier to fake a smaller bound | Mallory / rival clearing bot | **reverted** · `DualInfeasible` | `0xa7d7e7bc…ef8a` | 11949600 | 138,136 |
| N6 | Proven-maximal netting executes: 169.00 tUSD of debt cancelled, 200.00 → 31.00 | QED agent | success | `0x46288ece…05a2` | 11949601 | 550,826 |
| N7 | Replay the executed netting | Mallory / rival clearing bot | **reverted** · `IncompleteSnapshot` | `0x4588659b…4df5` | 11949602 | 138,136 |
| S1 | Northwind pays QED the remaining 18.00 tUSD | Northwind | success | `0x8e6ffd33…358c` | 11949687 | 190,980 |
| S2 | CloudCo pays Atlas the remaining 3.00 tUSD | CloudCo | success | `0x8be4b11f…8ebf` | 11949688 | 93,060 |
| S3 | Helix pays QED the remaining 4.00 tUSD | Helix | success | `0xe8af7d69…0648` | 11949689 | 93,060 |
| S4 | Atlas pays QED the remaining 6.00 tUSD | Atlas | success | `0xb69b59c9…a654` | 11949690 | 91,741 |

Final token balances equal each agent's net position: Northwind 0.00 · QED 28.00 · CloudCo 0.00 · Helix 0.00 · Atlas 0.00. Mallory received 0.

Set-up transactions of this run (funding, mints): `0xda9bfc81…7390`, `0x8e8c0c3f…f2b6`, `0xbb7e24a6…a102`, `0xd23466d2…5222`, `0xe0a46cd9…001a`, `0xd0ba9980…c4c9`, `0x0ce9e106…3182`, `0x3e69b56d…19f5`, `0x0dc85d8e…de39`, `0x5e9ad960…91dc`, `0xab31a916…1411`, `0x63e9af20…40d2`, `0xe2af5f53…991b`, `0x97a1b6f6…7320`.

## Run 3 — zero-knowledge optimality and theorem bounties (QED Private)

> `evidence/ethereum-sepolia/private/manifest.json`, `receipts.json`. Built on the main run's ProofEscrow, TestUSD and MinVarianceVerifier; `npm run verify` re-verifies the Groth16 proof off-chain with snarkjs, recovers the vendor's attestation from calldata and checks both bounty pools.

| | |
|---|---|
| Blocks | 11,951,192 → 11,951,215 |
| Transactions | 24 (16 succeeded · 8 reverted **by design**) |
| Private job | job #3: 12.00 tUSD paid into QED's vault on a Groth16 proof (12,569 constraints, proved in 1.5 s) |
| What the chain saw | data commitment `0x0ec573bb27cdf14a…` (attested by Helix), portfolio commitment `0x057717d386e492ea…`, variance 0.0015702689, provider |
| Bounties | 1000.00 + 1000.00 tUSD unclaimed after every refutation attempt |

| Contract | Address |
|---|---|
| Groth16Verifier | `0xc04329693c7b464ab636107b31dcf6746e204e7b` |
| PrivateMinVarianceVerifier | `0x1270287aaf21f6cc939c255fc1ba294d9268b453` |
| OptimalityBounty | `0xfc2129f0432538d034c5307299022b3d369eb677` |
| MaximalityBounty | `0xd767782fc47eefdfc2d764dba1b2ec0a0b557d69` |

| Role | Address |
|---|---|
| deployer | `0x94C938E642BbC657D5F45546E35268a70c936448` |
| client | `0x053DCBCaEF06A44feAeFd94A2DbE9F160E789816` |
| qed | `0x0460e0d0753E487b3a5E26FE8cf16A4baC6C918d` |
| helix | `0xE21F926030fd77Ad4EFEbd1b22D6633cda3499C7` |

| Role | Address |
|---|---|
| sponsor | `0x13958b9C4154996E6Db79f9a86c5765e0D6048AF` |
| mallory | `0x56D161255Ced6863698eA3c6a886982088fa30a8` |

| Step | What happened | Actor | Result | Transaction | Block | Gas |
|---|---|---|---|---|---|---|
| K3 | Deploy the Groth16 verifier generated from circuits/private_minvar.circom | Deployer | success | `0xe7edce9a…85ea` | 11951192 | 2,635,710 |
| K4 | Deploy PrivateMinVarianceVerifier (registered data vendor: Helix) | Deployer | success | `0xf28684ac…9868` | 11951193 | 3,420,169 |
| K5 | Deploy OptimalityBounty on the verifier that settled job #1 | Deployer | success | `0x2e91dbe5…cc1b` | 11951194 | 3,393,875 |
| K6 | Deploy MaximalityBounty on the clearing house's acceptance rule | Deployer | success | `0x487d5e0b…138a` | 11951195 | 4,400,401 |
| B1 | QED funds the optimality bounty: 1,000 tUSD | Deployer | success | `0x616b0fb1…3775` | 11951197 | 128,918 |
| B2 | Atlas funds the maximality bounty: 1,000 tUSD | Atlas | success | `0xdf74e4c8…d614` | 11951199 | 128,918 |
| V0 | Helix attests its Private Markets model by commitment only | Helix (data vendor) | off-chain | — | | |
| Q1 | QED quotes 12 tUSD for the provably optimal private portfolio; Mallory undercuts at 8 | QED agent / Mallory | off-chain | — | | |
| Z1 | Northwind funds job #3 in the same ProofEscrow: 12 tUSD, verifier = zero-knowledge | Northwind (client) | success | `0xed610de8…70b3` | 11951202 | 1,291,592 |
| Z2 | Northwind also funds Mallory's cheaper job #4: 8 tUSD | Northwind (client) | success | `0x8f0fe4d2…bf01` | 11951203 | 1,193,672 |
| P1 | QED proves the portfolio optimal in zero knowledge (1.5 s on one CPU) | QED agent | off-chain | — | | |
| Z3 | QED delivers the zero-knowledge proof: the escrow verifies it and pays 12 tUSD into QED's vault | QED agent | success | `0xbb619bf0…2ec7` | 11951204 | 530,050 |
| ZA1 | Mallory copies QED's proof from the public calldata into her own job | Mallory | **reverted** · `ProofRejected` | `0xbe69abd9…d28e` | 11951205 | 269,489 |
| ZA2 | Mallory claims a lower variance than any portfolio can achieve | Mallory | **reverted** · `ProofRejected` | `0x95426566…9431` | 11951206 | 269,501 |
| ZM | Mallory opens a self-dealt job #5 on a data set she attested herself | Mallory | success | `0xd721136c…9a44` | 11951209 | 1,190,784 |
| ZA3 | Mallory delivers a genuine proof about self-attested data | Mallory | **reverted** · `UntrustedDataAttestation` | `0x35bbaa47…25b7` | 11951210 | 62,360 |
| BA1 | Mallory claims the 1,000 tUSD bounty with the equal-weight portfolio against QED's job #1 certificate | Mallory | **reverted** · `TheoremHolds` | `0x6a584945…d19d` | 11951211 | 219,644 |
| BA2 | …and with QED's own optimum shifted by 0.1% | Mallory | **reverted** · `TheoremHolds` | `0xb9fbcad2…e059` | 11951212 | 219,344 |
| BA3 | …and with the forged 1/N certificate the chain refused in job #2 (X17) | Mallory | **reverted** · `DualityGapTooLarge` | `0x68e498bb…1c35` | 11951213 | 205,720 |
| BM1 | Mallory claims the clearing bounty with the naive netting (162 tUSD) | Mallory | **reverted** · `TheoremHolds` | `0x47ef93bf…62bb` | 11951214 | 195,480 |
| BM2 | …and with a 170 tUSD netting that cancels 1 tUSD more of Northwind's debt to QED | Mallory | **reverted** · `NetPositionChanged` | `0xd5124345…af2a` | 11951215 | 195,480 |

Set-up transactions of this run (faucet draws, approvals): `0xe8c1aa8a…8086`, `0xa6c497bf…ba06`, `0x0d750770…6e7a`, `0x52b9a61f…2552`, `0x1f93f3a3…93a7`, `0x37f9ba83…feb2`.

## How these transactions were produced — full disclosure

Signed in a build sandbox without RPC egress against a wall-clock-synchronised replica of Sepolia (same chain id, nonces, balances); the identical signed bytes were broadcast via a browser relay and every public receipt compared with the replica (status + logs). The final phase was executed after rebuilding the replica by replaying the exact signed transactions already on Sepolia.

- The relay fetched `relay/outbox-11155111.json`, `relay/outbox-11155111-b2.json`, `relay/outbox-11155111-clearing.json` and `relay/outbox-11155111-private.json` from a **pinned commit** of this repository and broadcast the bytes unchanged; the transaction hashes in this file are therefore the hashes of those exact bytes.
- Each public receipt was compared with the replica's receipt: 55/55 receipts matched the replica (status + logs; EIP-7708 system logs excluded); clearing run: 44/44 receipts matched the replica (status + logs; EIP-7708 system logs excluded); private run: 24/24 receipts matched.
- Block timestamps differ by seconds between replica and Sepolia; no assertion depends on them (windows are 24 h, certificates live 2 h, deadlines are days).
- Two earlier attempts under different keys failed and were abandoned: the first ran out of gas on deployment under Sepolia's state-creation repricing; in the second, one agent payment was under-provisioned with gas. The vault was slimmed (via-IR; views moved to QEDLens), every transaction now carries an explicit 16.7 M gas limit (only gas used is charged), and the canonical run above was executed under **fresh keys**, so it shares no state with those attempts.
- Gas used on Sepolia differs from the replica because Sepolia prices state creation and calldata differently; the relay compared status and logs, not gas.

## Block timestamps

First: block 11948657 at 2026-10-19T14:06:36Z · last: block 11949339 at 2026-10-19T16:24:36Z.